TURING 图灵原创

Python 3 网络爬虫开发实战

崔庆才 著


代理
Mitmproxy
HTML
验证码
Scrapy
式
Json
Response
Urllib
Redis
HTML
Scrapy
Appium
Charles
MongoDB
JavaScript
爬虫
证码
Request
Proxy
BloomFilter
Cookies
Selenium
Mitmproxy
Ajax
Urllib
PySpider
Regex
JavaScript
Response
Charles
Request
Redis
BloomFilter
Cookies
Response
Charles
Ajax
Urllib
Json
Proxy
Request
Regex
Selenium
Cookies
BloomFilter
PySpider
Mitmproxy
JavaScript
Response
Request
Proxy
Cookies
Selenium
Regex
Charles
Redis
Ajax
Urllib
Json
PySpider
BloomFilter
Proxy
Request
Ajax
Regex
Response
Selenium
Cookies
Charles
Redis
Appium
Mongodb
Scrapy
Request
Response
Proxy
Cookies
Selenium
Regex



崔庆才

北京航空航天大学硕士，静觅博客（https://cuiqingcai.com/）博主，爬虫博文访问量已过百万，喜欢钻研，热爱生活，乐于分享。欢迎关注个人微信公众号“进击的Coder”，二维码如下：

图灵原创

Python 3 网络爬虫开发实战

崔庆才 著

人民邮电出版社
北 京

图书在版编目（CIP）数据

Python 3网络爬虫开发实战 / 崔庆才著. -- 北京 : 人民邮电出版社, 2018.4
（图灵原创）
ISBN 978-7-115-48034-7

Ⅰ. ①P… Ⅱ. ①崔… Ⅲ. ①软件工具－程序设计 Ⅳ. ①TP311.561

中国版本图书馆CIP数据核字(2018)第042370号

内 容 提 要

本书介绍了如何利用 Python 3 开发网络爬虫。书中首先详细介绍了环境配置过程和爬虫基础知识；然后讨论了 urllib、requests 等请求库，Beautiful Soup、XPath、pyquery 等解析库以及文本和各类数据库的存储方法；接着通过多个案例介绍了如何进行 Ajax 数据爬取，如何使用 Selenium 和 Splash 进行动态网站爬取；再后介绍了爬虫的一些技巧，比如使用代理爬取和维护动态代理池的方法，ADSL 拨号代理的使用，图形、极验、点触、宫格等各类验证码的破解方法，模拟登录网站爬取的方法及 Cookies 池的维护。

此外，本书还结合移动互联网的特点探讨了使用 Charles、mitmdump、Appium 等工具实现 App 爬取的方法，紧接着介绍了 pyspider 框架和 Scrapy 框架的使用，以及分布式爬虫的知识，最后介绍了 Bloom Filter 效率优化、Docker 和 Scrapyd 爬虫部署、Gerapy 爬虫管理等方面的知识。

本书适合 Python 程序员阅读。

◆ 著　　崔庆才
责任编辑　王军花
责任印制　周昇亮
◆ 人民邮电出版社出版发行　　北京市丰台区成寿寺路11号
邮编　100164　　电子邮件　315@ptpress.com.cn
网址　http://www.ptpress.com.cn
大厂聚鑫印刷有限责任公司印刷
◆ 开本：800×1000　1/16
印张：37.75
字数：917千字　　2018年 4 月第 1 版
印数：1－4 000册　　2018年 4 月河北第 1 次印刷

定价：99.00元

读者服务热线：(010)51095186转600　印装质量热线：(010)81055316

反盗版热线：(010)81055315

广告经营许可证：京东工商广登字 20170147 号

序　一

人类社会已经进入大数据时代，大数据深刻改变着我们的工作和生活。随着互联网、移动互联网、社交网络等的迅猛发展，各种数量庞大、种类繁多、随时随地产生和更新的大数据，蕴含着前所未有的社会价值和商业价值。大数据成为 21 世纪最为重要的经济资源之一。正如马云所言：未来最大的能源不是石油而是大数据。对大数据的获取、处理与分析，以及基于大数据的智能应用，已成为提高未来竞争力的关键要素。

但如何获取这些宝贵数据呢？网络爬虫就是一种高效的信息采集利器，利用它可以快速、准确地采集我们想要的各种数据资源。因此，可以说，网络爬虫技术几乎已成为大数据时代 IT 从业者的必修课程。

我们需要采集的数据大多来源于互联网的各个网站。然而，不同的网站结构不一、布局复杂、渲染方式多样，有的网站还专门采取了一系列“反爬”的防范措施。因此，为准确高效地采集到需要的数据，我们需要采取具有针对性的反制措施。网络爬虫与反爬措施是矛与盾的关系，网络爬虫技术就是在这种针锋相对、见招拆招的不断斗争中，逐渐完善和发展起来的。

本书介绍了利用 Python 3 进行网络爬虫开发的各项技术，从环境配置、理论基础到进阶实战、分布式大规模采集，详细介绍了网络爬虫开发过程中需要了解的知识点，并通过多个案例介绍了不同场景下采用不同爬虫技术实现数据爬取的过程。

我坚信，每位读者学习和掌握了这些技术之后，成为一个爬虫高手将不再是梦想！

李舟军，北京航空航天大学教授，博士生导师

2017 年 10 月

序　二

众所周知，人工智能的这次浪潮和深度学习技术的突破密不可分，却很少有人会谈论另一位幕后英雄，即数据。如果不是网络上有如此多的图片，李飞飞教授也无法构建近千万的标注图片集合ImageNet，从而成就深度学习技术在图像识别领域的突破。如果不是在网络上有了如此多的聊天数据，小冰也不会学习到人类的情商，在聊天中带给人类惊喜、欢笑和抚慰。人工智能的进步离不开数据和算法的结合，人类无意间产生的数据却能够让机器学习到超乎想象的“智慧”，反过来服务人类。

在互联网时代，强大的爬虫技术造就了很多伟大的搜索引擎公司，让人类的记忆搜索能力得到巨大的延展。今天在移动互联网时代，爬虫技术仍然是支撑一些信息融合应用（如今日头条）的关键技术。但是，今天爬虫技术面临着更大的挑战。与互联网的共享机制不同，很多资源只有在登录之后才能访问，还采取了各种反爬虫措施，这就让爬虫不那么容易访问这些资源。无论是产品还是研究，都需要大量的优质数据来让机器更加智能。因此，在这个时代，大量的从业者急需一本全面介绍爬虫技术的书。如果你需要了解全面和前沿的爬虫技术，而且想迅速地上手实战，这本书就是首选。

我很荣幸认识崔庆才先生，他目前还是一名北京航空航天大学在读研究生，正处在一个对技术狂热追求的年纪。我听他讲了一些修炼爬虫技术的故事，很有意思。他在本科的时候因为一个项目开始接触爬虫，之后他用爬虫竟然得到了所在学校同学的照片，还帮助他的哥们儿追其他系的女孩。我问他是否也是用这些信息找到了女友，他甩了下头发，酷酷地说：“需要吗？”

崔庆才是个非常擅长学习的人，他玩什么都能玩到精通。他有一个很好的习惯，就是边学边写，他早期学习爬虫技术的时候，就开了博客，边学边分享他学到并实际操作过的经验，圈粉无数。我很受启发，这样的学习模式很高效，要教给别人之前自己必须弄得特别清楚。另一方面，互联网上的互动也给了他继续学习和精益求精的动力。

除了网络，图书是最成体系的经验分享。本书记录了崔庆才先生对爬虫实战技术最精华的部分。我已经迫不及待地想买一本，也一定会把它推荐给更多的朋友。

宋睿华，微软小冰首席科学家

2017 年 10 月

前　言

为什么写这本书

在这个大数据时代，尤其是人工智能浪潮兴起的时代，不论是工程领域还是研究领域，数据已经成为必不可少的一部分，而数据的获取很大程度上依赖于爬虫的爬取，所以爬虫也逐渐变得火爆起来。我是在 2015 年开始接触爬虫的，当时爬虫其实并没有这么火，我当时觉得能够把想要的数据抓取下来就是一件非常有成就感的事情，而且也可以顺便熟悉 Python，一举两得。在学习期间，我将学到的内容做好总结，发表到博客上。随着我发表的内容越来越多，博客的浏览量也越来越多，很多读者对我的博文给予了肯定的评价，这也给我的爬虫学习之路增添了很多动力。在学习的过程中，困难其实还是非常多的，最早学习时使用的是 Python 2，当时因为编码问题搞得焦头烂额。另外，那时候相关的中文资料还比较少，很多情况下还得自己慢慢去啃官方文档，走了不少弯路。随着学习的进行，我发现爬虫这部分内容涉及的知识点太多、太杂了。网页的结构、渲染方式不同，我们就得换不同的爬取方案来进行针对性的爬取。另外，网页信息的提取、爬取结果的保存也有五花八门的方案。随着移动互联网的兴起，App 的爬取也成了一个热点，而为了提高爬取速度又需要考虑并行爬取、分布式爬取方面的内容，爬虫的通用性、易用性、架构都需要好好优化。这么多杂糅的知识点对于一个爬虫初学者来说，学习的挑战性会非常高，同时学习过程中大家或许也会走我之前走过的弯路，浪费很多时间。后来有一天，图灵的王编辑联系了我，问我有没有意向写一本爬虫方面的书，我听到之后充满了欣喜和期待，这样既能把自己学过的知识点做一个系统整理，又可以跟广大爬虫爱好者分享自己的学习经验，还可以出版自己的作品，于是我很快就答应约稿了。

一开始觉得写书并不是一件那么难的事，后来真正写了才发现其中包含的艰辛。书相比博客来说，用词的严谨性要高很多，而且逻辑需要更加缜密，很多细节必须考虑得非常周全。前前后后写了大半年的时间，审稿和修改又花费了几个月的时间，一路走来甚是不易，不过最后看到书稿成型，觉得这一切都是值得的。在书中，我把我学习爬虫的很多经验都写了进去。环境配置是学习的第一步，环境配置不好，其他工作就没法开展，甚至可能很大程度上打击学习的积极性，所以我在第 1 章中着重介绍了环境的配置过程。而因为操作系统的不同，环境配置过程又各有不同，所以我把每个系统（Windows、Linux、Mac）的环境配置过程都亲自实践了一遍，并梳理记录下来，希望为各位读者在环境配置时多提供一些帮助。后面我又针对爬虫网站的不同情形分门别类地进行了说明，如 Ajax 分析爬取、动态渲染页面爬取、App 爬取、使用代理爬取、模拟登录爬取等知识，每个知识点我都选取了一些典型案例来说明，以便于读者更好地理解整个过程和用法。为了提高代码编写和爬取的效率，还可以使用一些爬虫框架辅助爬取，所以本书后面又介绍了两个流行的爬虫框架的用法，最后又介绍

了一些分布式爬虫及部署方面的知识。总体来说，本书根据我个人觉得比较理想的学习路径介绍了学习爬虫的相关知识，并通过一些实战案例帮助读者更好地理解其中的原理。

本书内容

本书一共分为 15 章，归纳如下。

- 第1章介绍了本书所涉及的所有环境的配置详细流程，兼顾Windows、Linux、Mac三大平台。本章不用逐节阅读，需要的时候查阅即可。
- 第2章介绍了学习爬虫之前需要了解的基础知识，如HTTP、爬虫、代理的基本原理、网页基本结构等内容，对爬虫没有任何了解的读者建议好好了解这一章的知识。
- 第3章介绍了最基本的爬虫操作，一般学习爬虫都是从这一步学起的。这一章介绍了最基本的两个请求库（urllib 和 requests）和正则表达式的基本用法。学会了这一章，就可以掌握最基本的爬虫技术了。
- 第 4 章介绍了页解析库的基本用法，包括 Beautiful Soup、XPath、pyquery 的基本使用方法，它们可以使得信息的提取更加方便、快捷，是爬虫必备利器。
- 第 5 章介绍了数据存储的常见形式及存储操作，包括 TXT、JSON、CSV 各种文件的存储，以及关系型数据库 MySQL 和非关系型数据库 MongoDB、Redis 存储的基本存储操作。学会了这些内容，我们可以灵活方便地保存爬取下来的数据。
- 第 6 章介绍了 Ajax 数据爬取的过程，一些网页的数据可能是通过 Ajax 请求 API 接口的方式加载的，用常规方法无法爬取，本章介绍了使用 Ajax 进行数据爬取的方法。
- 第 7 章介绍了动态渲染页面的爬取，现在越来越多的网站内容是经过 JavaScript 渲染得到的，而原始 HTML 文本可能不包含任何有效内容，而且渲染过程可能涉及某些 JavaScript 加密算法，可以使用 Selenium、Splash 等工具来实现模拟浏览器进行数据爬取的方法。
- 第8章介绍了验证码的相关处理方法。验证码是网站反爬虫的重要措施，我们可以通过本章了解到各类验证码的应对方案，包括图形验证码、极验验证码、点触验证码、微博宫格验证码的识别。
- 第 9 章介绍了代理的使用方法，限制 IP 的访问也是网站反爬虫的重要措施。另外，我们也可以使用代理来伪装爬虫的真实 IP，使用代理可以有效解决这个问题。通过本章，我们了解到代理的使用方法，还学习了代理池的维护方法，以及 ADSL 拨号代理的使用方法。
- 第 10 章介绍了模拟登录爬取的方法，某些网站需要登录才可以看到需要的内容，这时就需要用爬虫模拟登录网站再进行爬取了。本章介绍了最基本的模拟登录方法以及维护一个 Cookies 池的方法。
- 第 11 章介绍了 App 的爬取方法，包括基本的 Charles、mitmproxy 抓包软件的使用。此外，还介绍了 mitmdump 对接 Python 脚本进行实时抓取的方法，以及使用 Appium 完全模拟手机 App 的操作进行爬取的方法。
- 第 12 章介绍了 pyspider 爬虫框架及用法，该框架简洁易用、功能强大，可以节省大量开发爬虫的时间。本章结合案例介绍了使用该框架进行爬虫开发的方法。

- 第 13 章介绍了 Scrapy 爬虫框架及用法。Scrapy 是目前使用最广泛的爬虫框架，本章介绍了它的基本架构、原理及各个组件的使用方法，另外还介绍了 Scrapy 通用化配置、对接 Docker 的一些方法。
- 第 14 章介绍了分布式爬虫的基本原理及实现方法。为了提高爬取效率，分布式爬虫是必不可少的，本章介绍了使用 Scrapy 和 Redis 实现分布式爬虫的方法。
- 第 15 章介绍了分布式爬虫的部署及管理方法。方便快速地完成爬虫的分布式部署，可以节省开发者大量的时间。本章结合 Scrapy、Scrapyd、Docker、Gerapy 等工具介绍了分布式爬虫部署和管理的实现。

致谢

感谢我的父母、导师，没有他们创造的环境，我不可能完成此书的写作。

感谢我的女朋友李园，在我写书期间给了我很多的支持和鼓励。同时她还主导设计了本书的封面，正是她的理解和付出才使本书得以完善。

感谢在我学习过程中与我探讨技术的各位朋友，特别感谢汪海洋先生在我初学爬虫过程中给我提供的指导，特别感谢崔弦毅、苟桃、时猛先生在我写书过程中为我提供的思路和建议。

感谢为本书撰写推荐语的李舟军老师、宋睿华老师、梁斌老师、施水才老师（排名不分先后），感谢你们对本书的支持和推荐。

感谢王军花、陈兴璐编辑，在书稿的审核过程中给我提供了非常多的建议，没有你们的策划和敦促，我也难以顺利完成此书。

感谢为本书做出贡献的每一个人！

相关资源

本书中的所有代码都放在了 GitHub（详见 https://github.com/Python3WebSpider），书中每个实例对应的章节末也有说明。

本人的个人博客也会更新爬虫相关文章，欢迎读者访问交流，博客地址：https://cuiqingcai.com/。



崔庆才

2018 年 1 月

目　录

第 1 章　开发环境配置 ····· 1
1.1 Python 3 的安装 ····· 1
1.1.1 Windows 下的安装 ····· 1
1.1.2 Linux 下的安装 ····· 6
1.1.3 Mac 下的安装 ····· 8
1.2 请求库的安装 ····· 10
1.2.1 requests 的安装 ····· 10
1.2.2 Selenium 的安装 ····· 11
1.2.3 ChromeDriver 的安装 ····· 12
1.2.4 GeckoDriver 的安装 ····· 15
1.2.5 PhantomJS 的安装 ····· 17
1.2.6 aiohttp 的安装 ····· 18
1.3 解析库的安装 ····· 19
1.3.1 lxml 的安装 ····· 19
1.3.2 Beautiful Soup 的安装 ····· 21
1.3.3 pyquery 的安装 ····· 22
1.3.4 tesserocr 的安装 ····· 22
1.4 数据库的安装 ····· 26
1.4.1 MySQL 的安装 ····· 27
1.4.2 MongoDB 的安装 ····· 29
1.4.3 Redis 的安装 ····· 36
1.5 存储库的安装 ····· 39
1.5.1 PyMySQL 的安装 ····· 39
1.5.2 PyMongo 的安装 ····· 39
1.5.3 redis-py 的安装 ····· 40
1.5.4 RedisDump 的安装 ····· 40
1.6 Web 库的安装 ····· 41
1.6.1 Flask 的安装 ····· 41
1.6.2 Tornado 的安装 ····· 42
1.7 App 爬取相关库的安装 ····· 43
1.7.1 Charles 的安装 ····· 44
1.7.2 mitmproxy 的安装 ····· 50
1.7.3 Appium 的安装 ····· 55
1.8 爬虫框架的安装 ····· 59
1.8.1 pyspider 的安装 ····· 59
1.8.2 Scrapy 的安装 ····· 61
1.8.3 Scrapy-Splash 的安装 ····· 65
1.8.4 Scrapy-Redis 的安装 ····· 66
1.9 部署相关库的安装 ····· 67
1.9.1 Docker 的安装 ····· 67
1.9.2 Scrapyd 的安装 ····· 71
1.9.3 Scrapyd-Client 的安装 ····· 74
1.9.4 Scrapyd API 的安装 ····· 75
1.9.5 Scrapyrt 的安装 ····· 75
1.9.6 Gerapy 的安装 ····· 76
第 2 章　爬虫基础 ····· 77
2.1 HTTP 基本原理 ····· 77
2.1.1 URI 和 URL ····· 77
2.1.2 超文本 ····· 78
2.1.3 HTTP 和 HTTPS ····· 78
2.1.4 HTTP 请求过程 ····· 80
2.1.5 请求 ····· 82
2.1.6 响应 ····· 84
2.2 网页基础 ····· 87
2.2.1 网页的组成 ····· 87
2.2.2 网页的结构 ····· 88
2.2.3 节点树及节点间的关系 ····· 90
2.2.4 选择器 ····· 91
2.3 爬虫的基本原理 ····· 93
2.3.1 爬虫概述 ····· 93

2.3.2　能抓怎样的数据……94
2.3.3　JavaScript 渲染页面……94
2.4　会话和 Cookies……95
2.4.1　静态网页和动态网页……95
2.4.2　无状态 HTTP……96
2.4.3　常见误区……98
2.5　代理的基本原理……99
2.5.1　基本原理……99
2.5.2　代理的作用……99
2.5.3　爬虫代理……100
2.5.4　代理分类……100
2.5.5　常见代理设置……101

第 3 章　基本库的使用……102
3.1　使用 urllib……102
3.1.1　发送请求……102
3.1.2　处理异常……112
3.1.3　解析链接……114
3.1.4　分析 Robots 协议……119
3.2　使用 requests……122
3.2.1　基本用法……122
3.2.2　高级用法……130
3.3　正则表达式……139
3.4　抓取猫眼电影排行……150

第 4 章　解析库的使用……158
4.1　使用 XPath……158
4.2　使用 Beautiful Soup……168
4.3　使用 pyquery……184

第 5 章　数据存储……197
5.1　文件存储……197
5.1.1　TXT 文本存储……197
5.1.2　JSON 文件存储……199
5.1.3　CSV 文件存储……203
5.2　关系型数据库存储……207
5.2.1　MySQL 的存储……207
5.3　非关系型数据库存储……213
5.3.1　MongoDB 存储……214
5.3.2　Redis 存储……221

第 6 章　Ajax 数据爬取……232
6.1　什么是 Ajax……232
6.2　Ajax 分析方法……234
6.3　Ajax 结果提取……238
6.4　分析 Ajax 爬取今日头条街拍美图……242

第 7 章　动态渲染页面爬取……249
7.1　Selenium 的使用……249
7.2　Splash 的使用……262
7.3　Splash 负载均衡配置……286
7.4　使用 Selenium 爬取淘宝商品……289

第 8 章　验证码的识别……298
8.1　图形验证码的识别……298
8.2　极验滑动验证码的识别……301
8.3　点触验证码的识别……311
8.4　微博宫格验证码的识别……318

第 9 章　代理的使用……326
9.1　代理的设置……326
9.2　代理池的维护……333
9.3　付费代理的使用……347
9.4　ADSL 拨号代理……351
9.5　使用代理爬取微信公众号文章……364

第 10 章　模拟登录……379
10.1　模拟登录并爬取 GitHub……379
10.2　Cookies 池的搭建……385

第 11 章　App 的爬取……398
11.1　Charles 的使用……398
11.2　mitmproxy 的使用……405
11.3　mitmdump 爬取“得到”App 电子书信息……417
11.4　Appium 的基本使用……423
11.5　Appium 爬取微信朋友圈……433
11.6　Appium+mitmdump 爬取京东商品……437

第 12 章　pyspider 框架的使用……443
12.1　pyspider 框架介绍……443
12.2　pyspider 的基本使用……445

12.3 pyspider 用法详解 ····· 459

第 13 章 Scrapy 框架的使用 ····· 468

13.1 Scrapy 框架介绍 ····· 468
13.2 Scrapy 入门 ····· 470
13.3 Selector 的用法 ····· 480
13.4 Spider 的用法 ····· 486
13.5 Downloader Middleware 的用法 ····· 487
13.6 Spider Middleware 的用法 ····· 494
13.7 Item Pipeline 的用法 ····· 496
13.8 Scrapy 对接 Selenium ····· 506
13.9 Scrapy 对接 Splash ····· 511
13.10 Scrapy 通用爬虫 ····· 516
13.11 Scrapyrt 的使用 ····· 533
13.12 Scrapy 对接 Docker ····· 536
13.13 Scrapy 爬取新浪微博 ····· 541

第 14 章 分布式爬虫 ····· 555

14.1 分布式爬虫原理 ····· 555
14.2 Scrapy-Redis 源码解析 ····· 558
14.3 Scrapy 分布式实现 ····· 564
14.4 Bloom Filter 的对接 ····· 569

第 15 章 分布式爬虫的部署 ····· 577

15.1 Scrapyd 分布式部署 ····· 577
15.2 Scrapyd-Client 的使用 ····· 582
15.3 Scrapyd 对接 Docker ····· 583
15.4 Scrapyd 批量部署 ····· 586
15.5 Gerapy 分布式管理 ····· 590

第1章 开发环境配置



工欲善其事，必先利其器！

编写和运行程序之前，我们必须先把开发环境配置好。只有配置好了环境并且有了更方便的开发工具，我们才能更加高效地用程序实现相应的功能。然而很多情况下，我们可能在最开始就卡在环境配置上，如果这个过程花费了太多时间，学习的兴趣可能就下降了大半，所以本章专门对本书中所有的环境配置做一下说明。

本章将讲解书中使用的所有库及工具的安装过程。为了使书的条理更加清晰，本书将环境配置的过程统一合并为一章。本章不必逐节阅读，可以在需要的时候查阅。

在介绍安装过程时，我们会尽量兼顾各个平台。另外，书中也会指出一些常见的安装错误，以便快速高效地搭建好编程环境。

1.1 Python 3 的安装

既然要用 Python 3 开发爬虫，那么第一步一定是安装 Python 3。这里会介绍 Windows、Linux 和 Mac 三大平台下的安装过程。相关链接如下。

- 官方网站：http://python.org
- 下载地址：https://www.python.org/downloads
- 第三方库：https://pypi.python.org/pypi
- 官方文档：https://docs.python.org/3
- 中文教程：http://www.runoob.com/python3/python3-tutorial.html
- Awesome Python：https://github.com/vinta/awesome-python
- Awesome Python 中文版：https://github.com/jobbole/awesome-python-cn

1.1.1 Windows 下的安装

在 Windows 下安装 Python 3 的方式有两种。

- 一种是通过 Anaconda 安装，它提供了 Python 的科学计算环境，里面自带了 Python 以及常用的库。如果选用了这种方式，后面的环境配置方式会更加简便。
- 另一种是直接下载安装包安装，即标准的安装方式。

下面我们依次介绍这两种安装方式，任选其一即可。

1. Anaconda 安装

Anaconda 的官方下载链接为 https://www.continuum.io/downloads，选择 Python 3 版本的安装包下载即可，如图 1-1 所示。

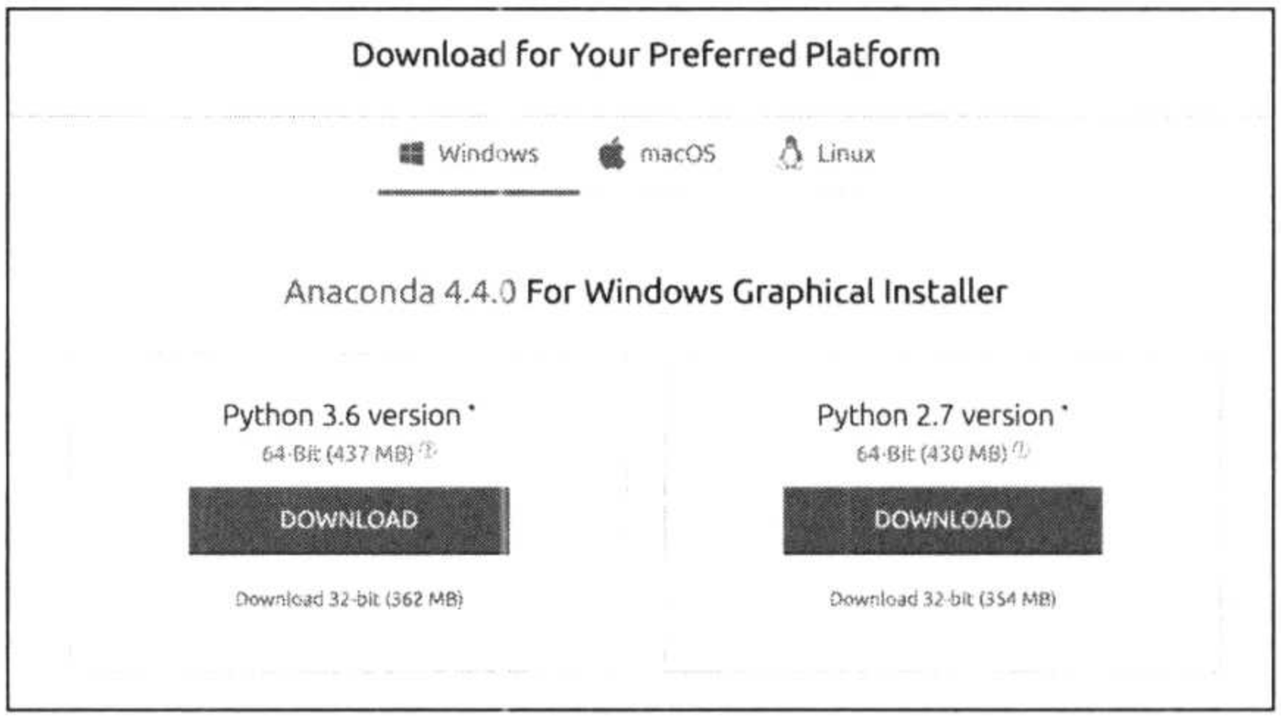


图 1-1　Anaconda Windows 下载页面

如果下载速度过慢，可以选择使用清华大学镜像，下载列表链接为 https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/，使用说明链接为 https://mirrors.tuna.tsinghua.edu.cn/help/anaconda/。

下载完成之后，直接双击安装包安装即可。安装完成之后，Python 3 的环境就配置好了。

2. 安装包安装

我们推荐直接下载安装包来安装，此时可以直接到官方网站下载 Python 3 的安装包：https://www.python.org/downloads/。

写书时，Python 的最新版本[①]是 3.6.2，其下载链接为 https://www.python.org/downloads/release/python-362/，下载页面如图 1-2 所示。需要说明的是，实际的 Python 最新版本以官网为准。

Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		2d0fc9f3a5940707590e07f03ecb08b9	22540566	SIG
XZ compressed source tarball	Source release		692b4fc3a2ba0d54d1495d4ead5b0b5c	16872064	SIG
Mac OS X 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6dd08e7027d2a1b3a2c02cfacbe611ef	27511848	SIG
Windows help file	Windows		69082441d723060fb333dcda8815105e	7986690	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64, not Itanium processors	708496ebbe9a730d19d5d288afd216f1	6926999	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64, not Itanium processors	ad69fdacde90f2ce8286c279b11ca188	31392272	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64, not Itanium processors	a055a1a0e938e74c712a1c495261ae6c	1312520	SIG
Windows x86 embeddable zip file	Windows		8dff09a1b19b7a7dcb915765328484cf	6320763	SIG
Windows x86 executable installer	Windows		3773db079c173bd6d8a631896c72a88f	30453192	SIG
Windows x86 web-based installer	Windows		f58f019335f39e0b45a0ae68027888d7	1287064	SIG

图 1-2　Python 下载页面

① 若无特别说明，书中的最新版本均为作者写书时的情况，后面不再一一说明。

64 位系统可以下载 Windows x86-64 executable installer，32 位系统可以下载 Windows x86 executable installer。

下载完成之后，直接双击 Python 安装包，然后通过图形界面安装，接着设置 Python 的安装路径，完成后将 Python 3 和 Python 3 的 Scripts 目录配置到环境变量即可。

关于环境变量的配置，此处以 Windows 10 系统为例进行演示。

假如安装后的 Python 3 路径为 C:\Python36，从资源管理器中打开该路径，如图 1-3 所示。

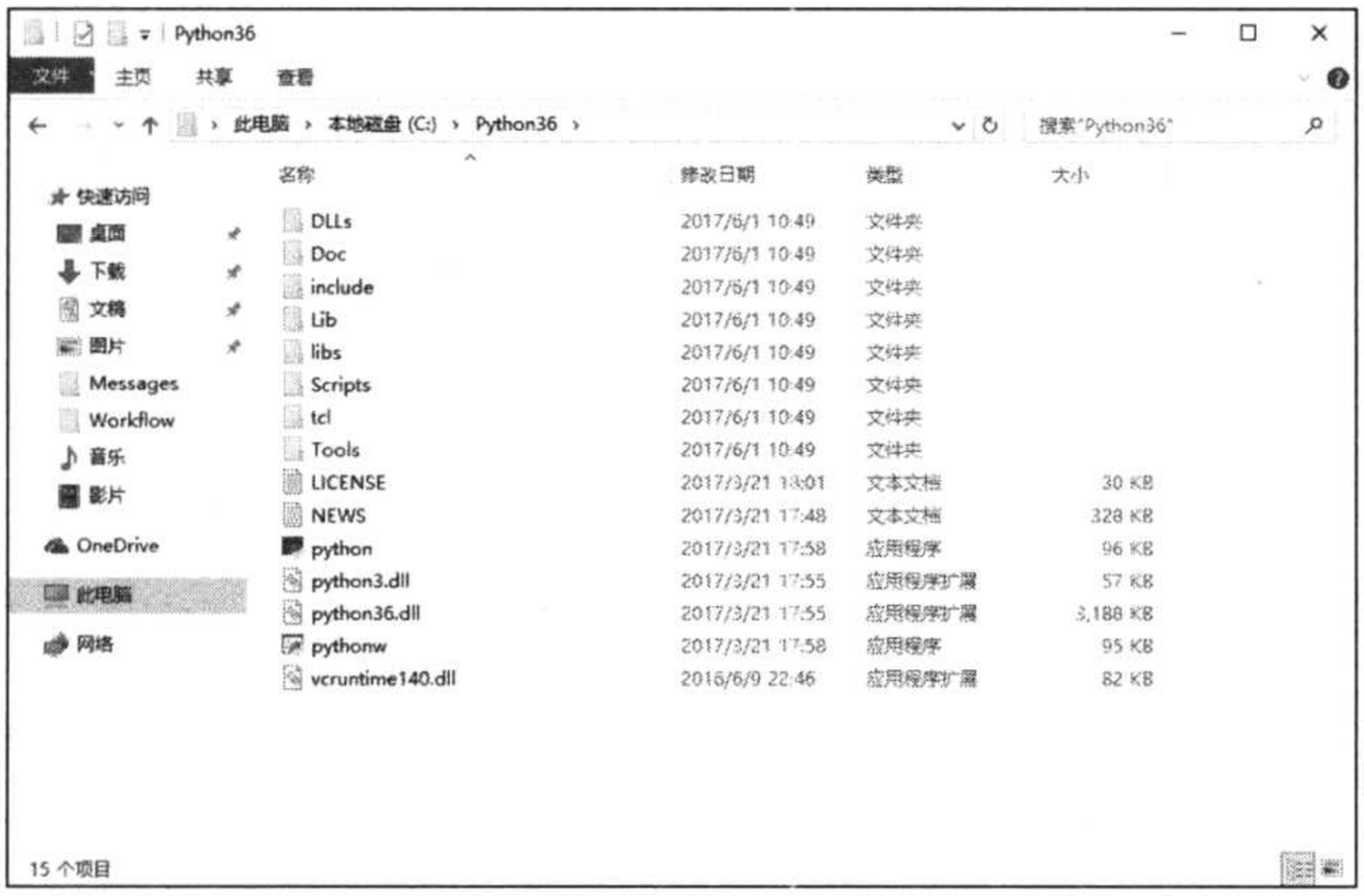


图 1-3 Python 安装目录

将该路径复制下来。

随后，右击“计算机”，从中选择“属性”，此时将打开系统属性窗口，如图 1-4 所示。



图 1-4 系统属性

点击左侧的“高级系统设置”，即可在弹出的对话框下方看到“环境变量”按钮，如图 1-5 所示。点击“环境变量”按钮，找到系统变量下的 Path 变量，随后点击“编辑”按钮，如图 1-6 所示。

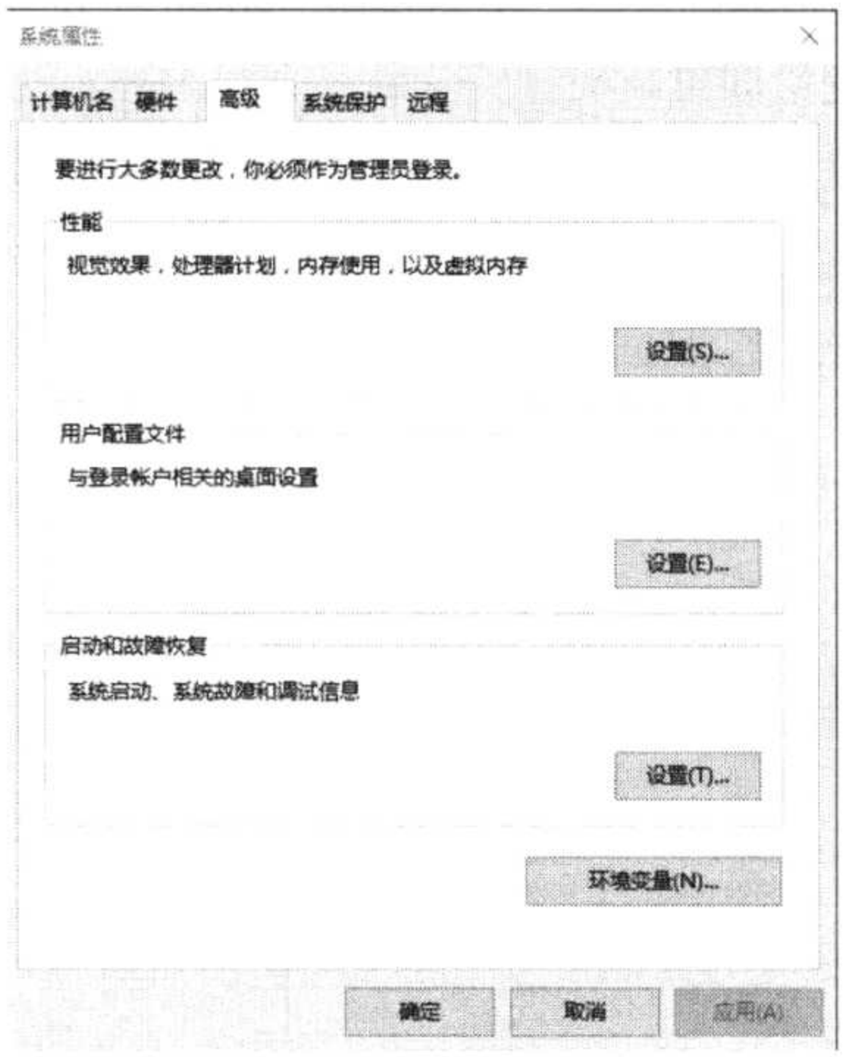


图 1-5　高级系统设置



图 1-6　环境变量

随后点击“新建”，新建一个条目，将刚才的 C:\Python36 复制进去。这里需要说明的是，此处的路径就是你的 Python 3 安装目录，请自行替换。然后，再把 C:\Python36\Scripts 路径复制进去，如图 1-7 所示。



图 1-7　编辑环境变量

最后，点击“确定”按钮即可完成环境变量的配置。

配置好环境变量后，我们就可以在命令行中直接执行环境变量路径下的可执行文件了，如 python、pip 等命令。

3. 添加别名

上面这两种安装方式任选其一即可完成安装，但如果之前安装过 Python 2 的话，可能会导致版本冲突问题，比如在命令行下输入 python 就不知道是调用的 Python 2 还是 Python 3 了。为了解决这个问题，建议将安装目录中的 python.exe 复制一份，命名为 python3.exe，这样便可以调用 python3 命令了。实际上，它和 python 命令是完全一致的，这样只是为了可以更好地区分 Python 版本。当然，如果没有安装过 Python 2 的话，也建议添加此别名，添加完毕之后的效果如图 1-8 所示。

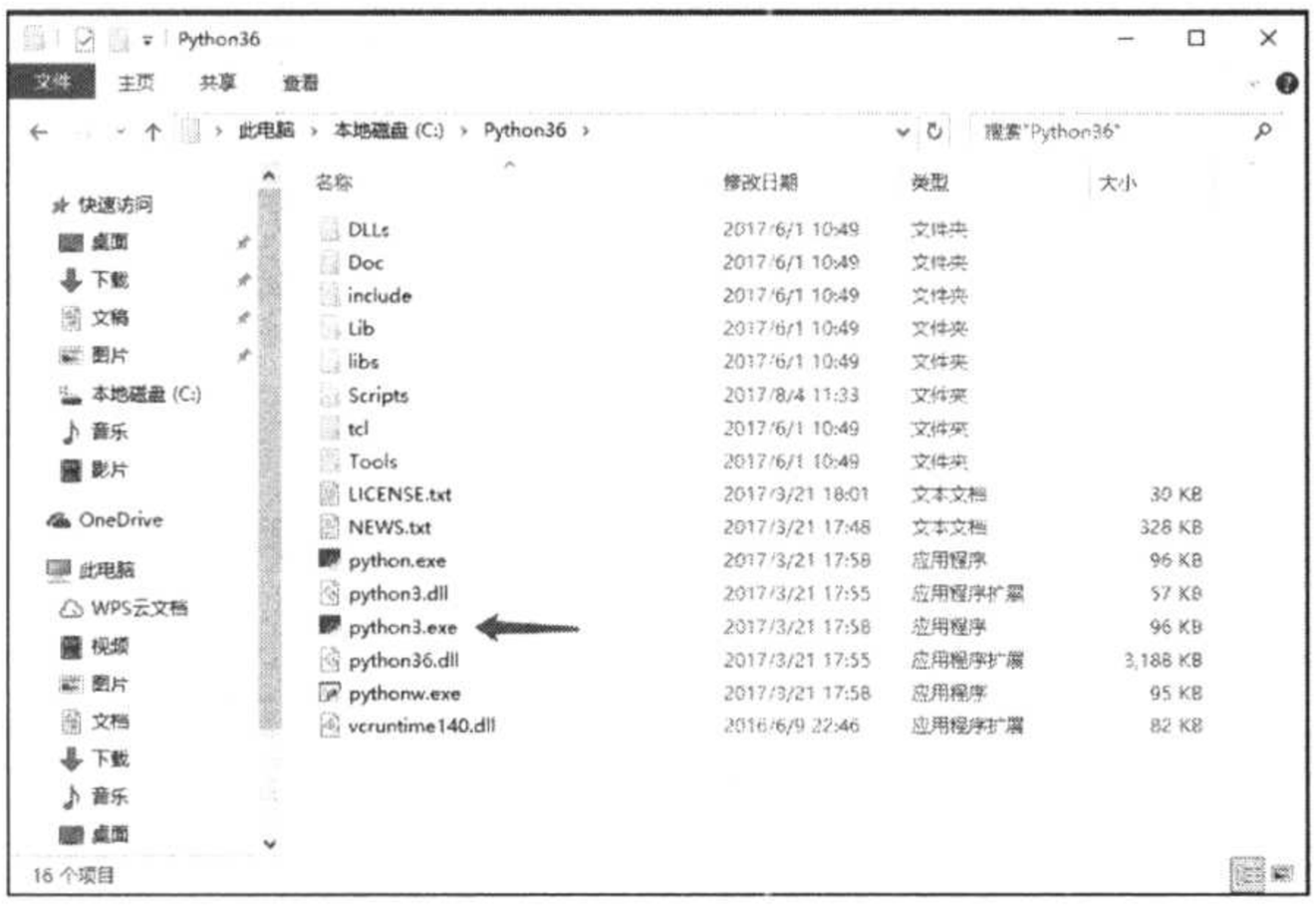


图 1-8 添加别名

对于 pip 来说，安装包中自带了 pip3.exe 可执行文件，我们也可以直接使用 pip3 命令，无需额外配置。

4. 测试验证

安装完成后，可以通过命令行测试一下安装是否成功。在“开始”菜单中搜索 cmd，找到命令提示符，此时就进入命令行模式了。输入 python，测试一下能否成功调用 Python。如果添加了别名的话，可以输入 python3 测试。这里输入的是 python3，测试结果如图 1-9 所示。

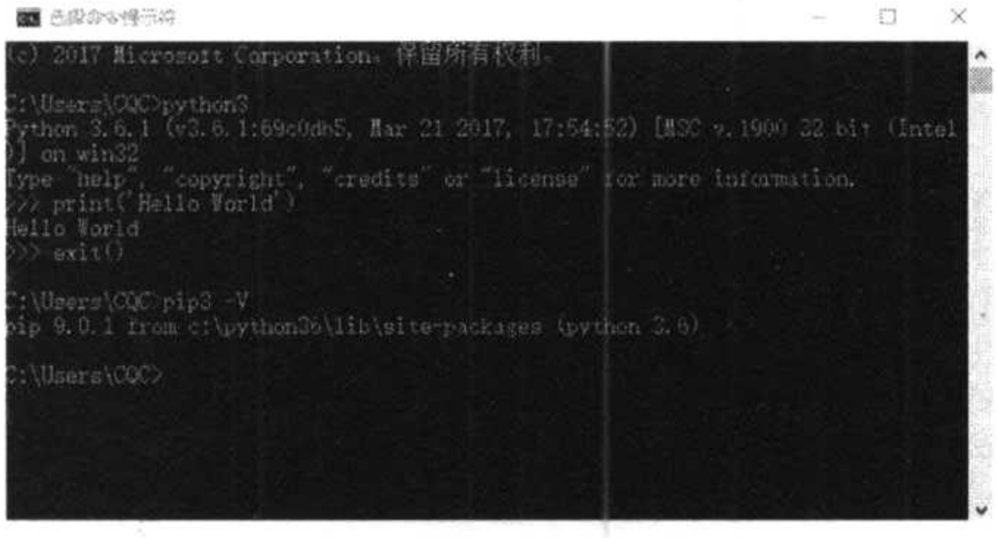


图 1-9 测试验证页面

输出结果类似如下：

```
$ python3
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print('Hello World')
Hello World
>>> exit()
$ pip3 -V
pip 9.0.1 from c:\python36\lib\site-packages (python 3.6)
```

如果出现了类似上面的提示，则证明 Python 3 和 pip 3 均安装成功；如果提示命令不存在，那么请检查下环境变量的配置情况。

1.1.2　Linux 下的安装

Linux 下的安装方式有多种：命令安装、源码安装和 Anaconda 安装。

使用源码安装需要自行编译，时间较长。推荐使用系统自带的命令或 Anaconda 安装，简单、高效。这里分别讲解这 3 种安装方式。

1. 命令行安装

不同的 Linux 发行版本的安装方式又有不同，在此分别予以介绍。

- **CentOS、Red Hat**

如果是 CentOS 或 Red Hat 版本，则使用 yum 命令安装即可。

下面列出了 Python 3.5 和 Python 3.4 两个版本的安装方法，可以自行选择。

Python 3.5 版本：

```
sudo yum install -y https://centos7.iuscommunity.org/ius-release.rpm
sudo yum update
sudo yum install -y python35u python35u-libs python35u-devel python35u-pip
```

执行完毕后，便可以成功安装 Python 3.5 及 pip 3 了。

Python 3.4 版本：

```
sudo yum groupinstall -y development tools
sudo yum install -y epel-release python34-devel  libxslt-devel libxml2-devel openssl-devel
sudo yum install -y python34
sudo yum install -y python34-setuptools
sudo easy_install-3.4 pip
```

执行完毕后，便可以成功安装 Python 3.4 及 pip 3 了。

- **Ubuntu、Debian 和 Deepin**

首先安装 Python 3，这里使用 apt-get 安装即可。在安装前，还需安装一些基础库，相关命令如下：

```
sudo apt-get install -y python3-dev build-essential libssl-dev libffi-dev libxml2 libxml2-dev libxslt1-dev
    zlib1g-dev libcurl4-openssl-dev
sudo apt-get install -y python3
```

执行完上述命令后，就可以成功安装 Python 3 了。

然后还需要安装 pip 3，这里仍然使用 apt-get 安装即可，相关命令如下：

```
sudo apt-get install -y python3-pip
```

执行完毕后，便可以成功安装 Python 3 及 pip 3 了。

2. 源码安装

如果命令行的安装方式有问题，还可以下载 Python 3 源码进行安装。

源码下载地址为 https://www.python.org/ftp/python/，可以自行选用想要的版本进行安装。这里以 Python 3.6.2 为例进行说明，安装路径设置为/usr/local/python3。

首先，创建安装目录，相关命令如下：

```
sudo mkdir /usr/local/python3
```

随后下载安装包并解压进入，相关命令如下：

```
wget --no-check-certificate https://www.python.org/ftp/python/3.6.2/Python-3.6.2.tgz
tar -xzvf Python-3.6.2.tgz
cd Python-3.6.2
```

接下来，编译安装。所需的时间可能较长，请耐心等待，命令如下：

```
sudo ./configure --prefix=/usr/local/python3
sudo make
sudo make install
```

安装完成之后，创建 Python 3 链接，相关命令如下：

```
sudo ln -s /usr/local/python3/bin/python3 /usr/bin/python3
```

随后下载 pip 安装包并安装，命令如下：

```
wget --no-check-certificate https://github.com/pypa/pip/archive/9.0.1.tar.gz
tar -xzvf 9.0.1.tar.gz
cd pip-9.0.1
python3 setup.py install
```

安装完成后再创建 pip 3 链接，相关命令如下：

```
sudo ln -s /usr/local/python3/bin/pip /usr/bin/pip3
```

这样就成功安装好了 Python 3 及 pip 3。

3. Anaconda 安装

Anaconda 同样支持 Linux，其官方下载链接为 https://www.continuum.io/downloads，选择 Python 3 版本的安装包下载即可，如图 1-10 所示。



图 1-10　Anaconda Linux 下载页面

如果下载速度过慢，同样可以使用清华镜像，具体可参考 Windows 部分的介绍，在此不再赘述。

4. 测试验证

在命令行界面下测试 Python 3 和 pip 3 是否安装成功：

```
$ python3
Python 3.5.2 (default, Nov 17 2016, 17:05:23)
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()
$ pip3 -V
pip 8.1.1 from /usr/lib/python3/dist-packages (python 3.5)
```

若出现类似上面的提示，则证明 Python 3 和 pip 3 安装成功。

1.1.3　Mac 下的安装

在 Mac 下同样有多种安装方式，如 Homebrew、安装包安装、Anaconda 安装等，这里推荐使用 Homebrew 安装。

1. Homebrew 安装

Homebrew 是 Mac 平台下强大的包管理工具，其官方网站是 https://brew.sh/。

执行如下命令，即可安装 Homebrew：

```
ruby -e "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install)"
```

安装完成后，便可以使用 brew 命令安装 Python 3 和 pip 3 了：

```
brew install python3
```

命令执行完成后，我们发现 Python 3 和 pip 3 均已成功安装。

2. 安装包安装

可以到官方网站下载 Python 3 安装包。链接为 https://www.python.org/downloads/，页面如图 1-2 所示。

在 Mac 平台下，可以选择下载 Mac OS X 64-bit/32-bit installer，下载完成后，打开安装包按照提示安装即可。

3. Anaconda 安装

Anaconda 同样支持 Mac，其官方下载链接为：https://www.continuum.io/downloads，选择 Python 3 版本的安装包下载即可，如图 1-11 所示。

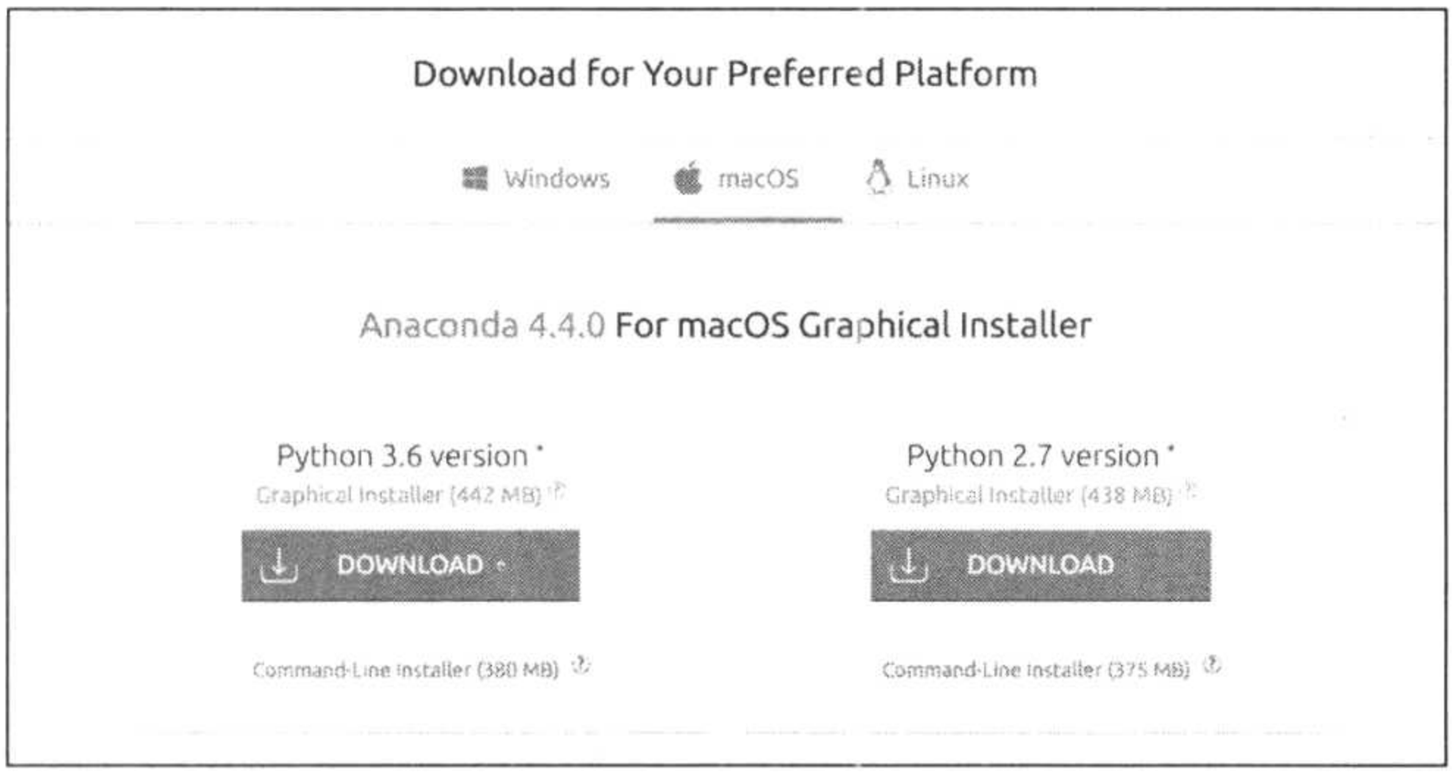


图 1-11 Anaconda Mac 下载页面

如果下载速度过慢，同样可以使用清华镜像，具体可参考 Windows 部分的介绍，在此不再赘述。

4. 测试验证

打开终端，在命令行界面中测试 Python 3 和 pip 3 是否成功安装，如图 1-12 所示。

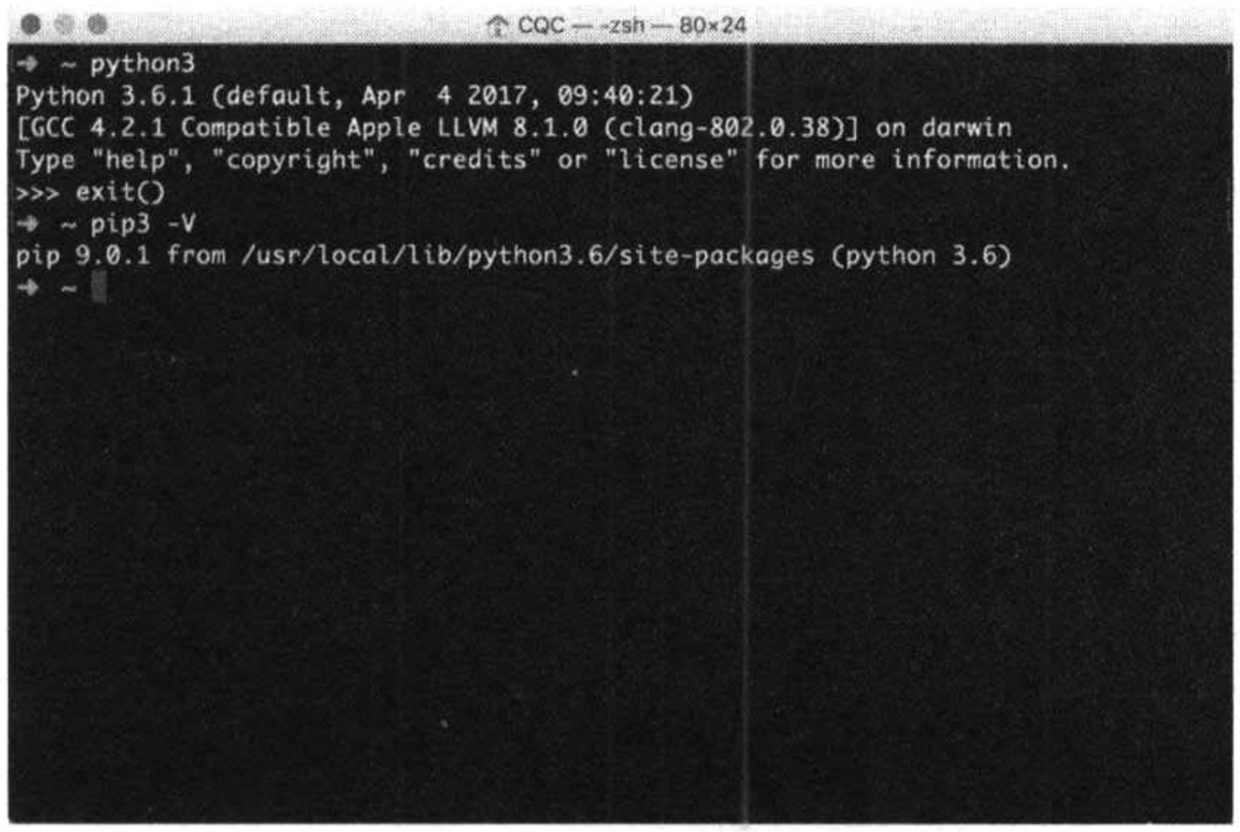


图 1-12 测试验证页面

若出现上面的提示，则证明 Python 3 和 pip 3 安装成功。

本节中，我们介绍了 3 大平台 Windows、Linux 和 Mac 下 Python 3 的安装方式。安装完成后，我们便可以开启 Python 爬虫的征程了。

1.2　请求库的安装

爬虫可以简单分为几步：抓取页面、分析页面和存储数据。

在抓取页面的过程中，我们需要模拟浏览器向服务器发出请求，所以需要用到一些 Python 库来实现 HTTP 请求操作。在本书中，我们用到的第三方库有 requests、Selenium 和 aiohttp 等。

在本节中，我们介绍一下这些请求库的安装方法。

1.2.1　requests 的安装

由于 requests 属于第三方库，也就是 Python 默认不会自带这个库，所以需要我们手动安装。下面我们首先看一下它的安装过程。

1. 相关链接

- GitHub：https://github.com/requests/requests
- PyPI：https://pypi.python.org/pypi/requests
- 官方文档：http://www.python-requests.org
- 中文文档：http://docs.python-requests.org/zh_CN/latest

2. pip 安装

无论是 Windows、Linux 还是 Mac，都可以通过 pip 这个包管理工具来安装。

在命令行界面中运行如下命令，即可完成 requests 库的安装：

```
pip3 install requests
```

这是最简单的安装方式，推荐使用这种方法安装。

3. wheel 安装

wheel 是 Python 的一种安装包，其后缀为.whl，在网速较差的情况下可以选择下载 wheel 文件再安装，然后直接用 pip3 命令加文件名安装即可。

不过在这之前需要先安装 wheel 库，安装命令如下：

```
pip3 install wheel
```

然后到 PyPI 上下载对应的 wheel 文件，如最新版本为 2.17.3，则打开 https://pypi.python.org/pypi/requests/2.17.3#downloads，下载 requests-2.17.3-py2.py3-none-any.whl 到本地。

随后在命令行界面进入 wheel 文件目录，利用 pip 安装即可：

```
pip3 install requests-2.17.3-py2.py3-none-any.whl
```

这样我们也可以完成 requests 的安装。

4. 源码安装

如果你不想用 pip 来安装，或者想获取某一特定版本，可以选择下载源码安装。

此种方式需要先找到此库的源码地址，然后下载下来再用命令安装。

requests 项目的地址是：https://github.com/kennethreitz/requests。

可以通过 Git 来下载源代码：

```
git clone git://github.com/kennethreitz/requests.git
```

或通过 curl 下载：

```
curl -OL https://github.com/kennethreitz/requests/tarball/master
```

下载下来之后，进入目录，执行如下命令即可安装：

```
cd requests
python3 setup.py install
```

命令执行结束后即可完成 requests 的安装。由于这种安装方式比较烦琐，后面不再赘述。

5. 验证安装

为了验证库是否已经安装成功，可以在命令行模式测试一下：

```
$ python3
>>> import requests
```

首先输入 python3，进入命令行模式，然后输入上述内容，如果什么错误提示也没有，就证明已经成功安装了 requests。

1.2.2 Selenium 的安装

Selenium 是一个自动化测试工具，利用它我们可以驱动浏览器执行特定的动作，如点击、下拉等操作。对于一些 JavaScript 渲染的页面来说，这种抓取方式非常有效。下面我们来看看 Selenium 的安装过程。

1. 相关链接

- 官方网站：http://www.seleniumhq.org
- GitHub：https://github.com/SeleniumHQ/selenium/tree/master/py
- PyPI：https://pypi.python.org/pypi/selenium
- 官方文档：http://selenium-python.readthedocs.io
- 中文文档：http://selenium-python-zh.readthedocs.io

2. pip 安装

这里推荐直接使用 pip 安装，执行如下命令即可：

```
pip3 install selenium
```

3. wheel 安装

此外，也可以到 PyPI 下载对应的 wheel 文件进行安装（下载地址：https://pypi.python.org/pypi/selenium/#downloads），如最新版本为 3.4.3，则下载 selenium-3.4.3-py2.py3-none-any.whl 即可。

然后进入 wheel 文件目录，使用 pip 安装：

```
pip3 install selenium-3.4.3-py2.py3-none-any.whl
```

4. 验证安装

进入 Python 命令行交互模式，导入 Selenium 包，如果没有报错，则证明安装成功：

```
$ python3
>>> import selenium
```

但这样做还不够，因为我们还需要用浏览器（如 Chrome、Firefox 等）来配合 Selenium 工作。

后面我们会介绍 Chrome、Firefox、PhantomJS 三种浏览器的配置方式。有了浏览器，我们才可以配合 Selenium 进行页面的抓取。

1.2.3　ChromeDriver 的安装

前面我们成功安装好了 Selenium 库，但是它是一个自动化测试工具，需要浏览器来配合使用，本节中我们就介绍一下 Chrome 浏览器及 ChromeDriver 驱动的配置。

首先，下载 Chrome 浏览器，方法有很多，在此不再赘述。

随后安装 ChromeDriver。因为只有安装 ChromeDriver，才能驱动 Chrome 浏览器完成相应的操作。下面我们来介绍下怎样安装 ChromeDriver。

1. 相关链接

- 官方网站：https://sites.google.com/a/chromium.org/chromedriver
- 下载地址：https://chromedriver.storage.googleapis.com/index.html

2. 准备工作

在这之前请确保已经正确安装好了 Chrome 浏览器并可以正常运行，安装过程不再赘述。

3. 查看版本

点击 Chrome 菜单“帮助”→“关于 Google Chrome”，即可查看 Chrome 的版本号，如图 1-13 所示。



图 1-13　Chrome 版本号

这里我的 Chrome 版本是 58.0。

请记住 Chrome 版本号，因为选择 ChromeDriver 版本时需要用到。

4. 下载 ChromeDriver

打开 ChromeDriver 的官方网站，可以看到最新版本为 2.31，其支持的 Chrome 浏览器版本为 58~60，官网页面如图 1-14 所示。

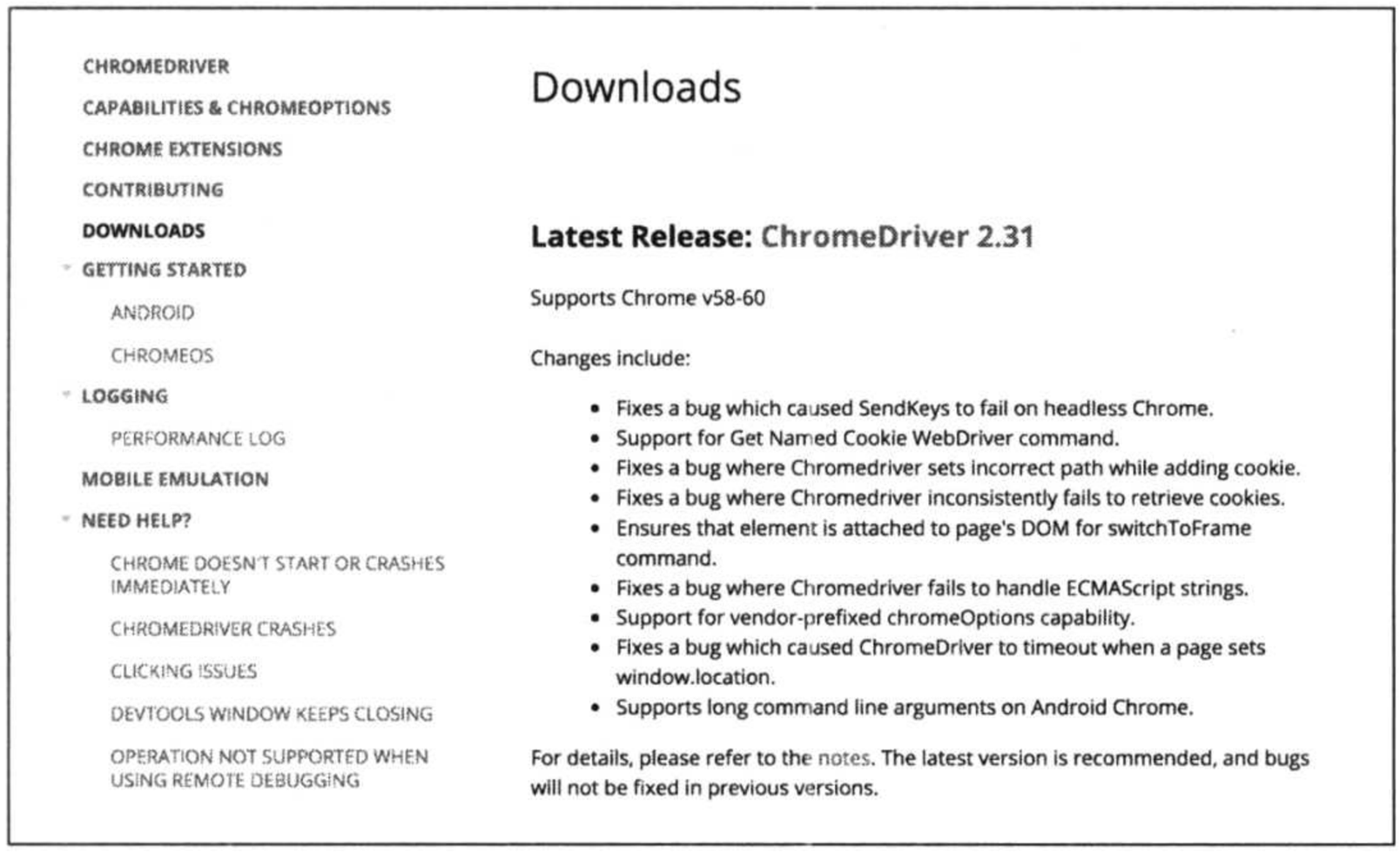


图 1-14 官网页面

如果你的 Chrome 版本号是 58~60，那么可以选择此版本下载。

如果你的 Chrome 版本号不在此范围，可以继续查看之前的 ChromeDriver 版本。每个版本都有相应的支持 Chrome 版本的介绍，请找好自己的 Chrome 浏览器版本对应的 ChromeDriver 版本再下载，否则可能无法正常工作。

找好对应的版本号后，随后到 ChromeDriver 镜像站下载对应的安装包即可：https://chromedriver.storage.googleapis.com/index.html。在不同平台下，可以下载不同的安装包。

5. 环境变量配置

下载完成后，将 ChromeDriver 的可执行文件配置到环境变量下。

在 Windows 下，建议直接将 chromedriver.exe 文件拖到 Python 的 Scripts 目录下，如图 1-15 所示。

此外，也可以单独将其所在路径配置到环境变量，具体的配置方法请参见 1.1 节。

在 Linux 和 Mac 下，需要将可执行文件配置到环境变量或将文件移动到属于环境变量的目录里。

例如，要移动文件到/usr/bin 目录。首先，需要在命令行模式下进入其所在路径，然后将其移动到/usr/bin：

```
sudo mv chromedriver /usr/bin
```



图 1-15　Python Scripts 目录

当然，也可以将 ChromeDriver 配置到$PATH。首先，可以将可执行文件放到某一目录，目录可以任意选择，例如将当前可执行文件放在/usr/local/chromedriver 目录下，接下来可以修改~/.profile 文件，相关命令如下：

```
export PATH="$PATH:/usr/local/chromedriver"
```

保存后执行如下命令：

```
source ~/.profile
```

即可完成环境变量的添加。

6. 验证安装

配置完成后，就可以在命令行下直接执行 chromedriver 命令了：

```
chromedriver
```

如果输入控制台有类似图 1-16 所示的输出，则证明 ChromeDriver 的环境变量配置好了。



图 1-16　控制台输出

随后再在程序中测试。执行如下 Python 代码：

```
from selenium import webdriver
browser = webdriver.Chrome()
```

运行之后，如果弹出一个空白的 Chrome 浏览器，则证明所有的配置都没有问题。如果没有弹出，请检查之前的每一步配置。

如果弹出后闪退，则可能是 ChromeDriver 版本和 Chrome 版本不兼容，请更换 ChromeDriver 版本。

如果没有问题，接下来就可以利用 Chrome 来做网页抓取了。

1.2.4 GeckoDriver 的安装

上一节中，我们了解了 ChromeDriver 的配置方法，配置完成之后便可以用 Selenium 驱动 Chrome 浏览器来做相应网页的抓取。

那么对于 Firefox 来说，也可以使用同样的方式完成 Selenium 的对接，这时需要安装另一个驱动 GeckoDriver。

本节中，我们来介绍一下 GeckoDriver 的安装过程。

1. 相关链接

- GitHub：https://github.com/mozilla/geckodriver
- 下载地址：https://github.com/mozilla/geckodriver/releases

2. 准备工作

在这之前请确保已经正确安装好了 Firefox 浏览器并可以正常运行，安装过程不再赘述。

3. 下载 GeckoDriver

我们可以在 GitHub 上找到 GeckoDriver 的发行版本，当前最新版本为 0.18，下载页面如图 1-17 所示。

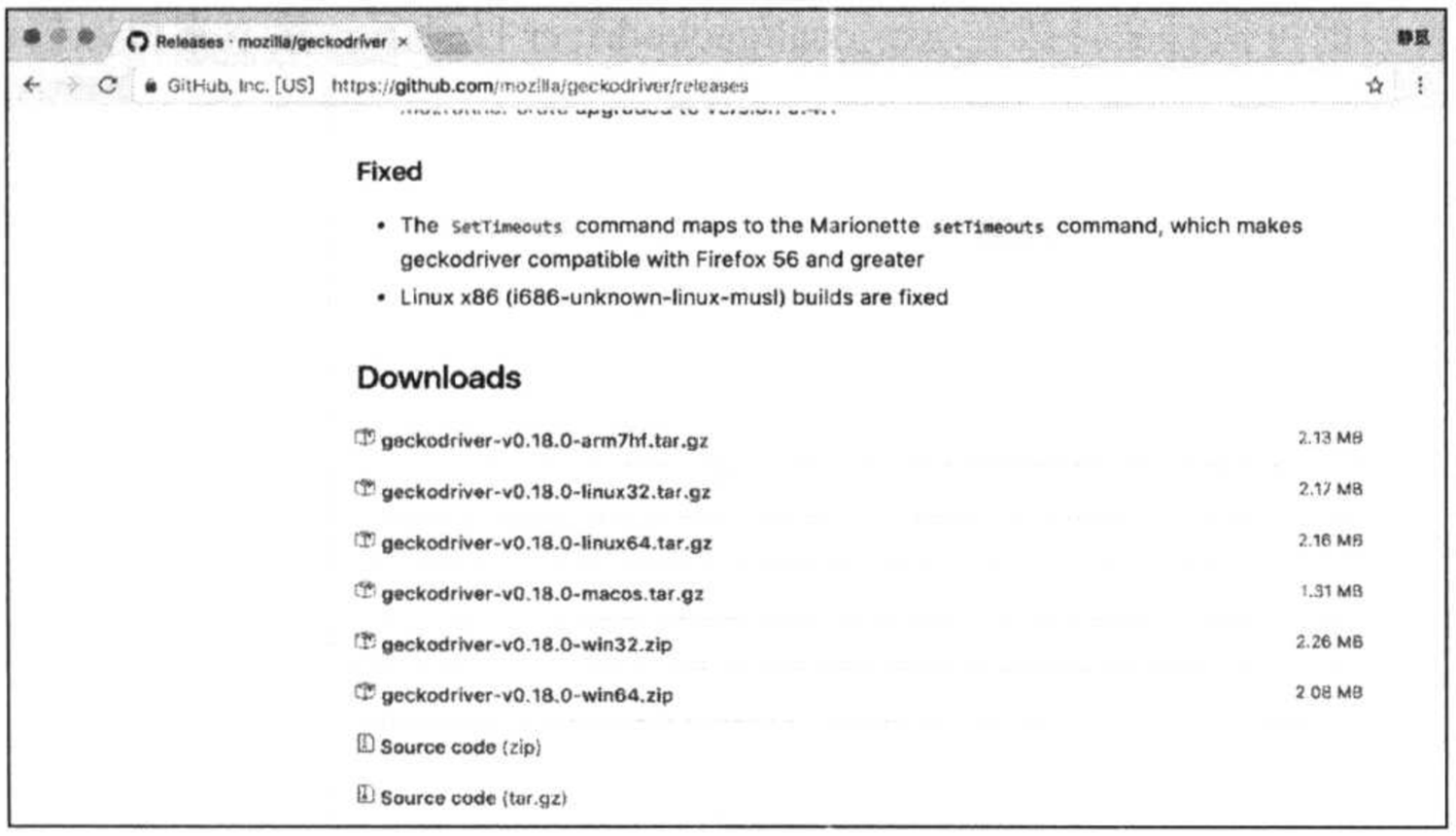


图 1-17 GeckoDriver 下载页面

这里可以在不同的平台上下载，如 Windows、Mac、Linux、ARM 等平台，我们可以根据自己的系统和位数选择对应的驱动下载，若是 Windows 64 位，就下载 geckodriver-v0.18.0-win64.zip。

4. 环境变量配置

在 Windows 下，可以直接将 geckodriver.exe 文件拖到 Python 的 Scripts 目录下，如图 1-18 所示。



图 1-18　将 geckodriver.exe 文件拖到 Python Scripts 目录

此外，也可以单独将其所在路径配置到环境变量，具体的配置方法请参 1.1 节。

在 Linux 和 Mac 下，需要将可执行文件配置到环境变量或将文件移动到属于环境变量的目录里。

例如，要移动文件到/usr/bin 目录。首先在命令行模式下进入其所在路径，然后将其移动到/usr/bin：

```
sudo mv geckodriver /usr/bin
```

当然，也可以将 GeckoDriver 配置到$PATH。首先，可以将可执行文件放到某一目录，目录可以任意选择，例如将当前可执行文件放在/usr/local/geckodriver 目录下。接下来可以修改~/.profile 文件，命令如下：

```
vi ~/.profile
```

然后添加如下一句配置：

```
export PATH="$PATH:/usr/local/geckodriver"
```

保存后执行如下命令即可完成配置：

```
source ~/.profile
```

5. 验证安装

配置完成后，就可以在命令行下直接执行 geckodriver 命令测试：

```
geckodriver
```

这时如果控制台有类似图 1-19 所示的输出，则证明 GeckoDriver 的环境变量配置好了。

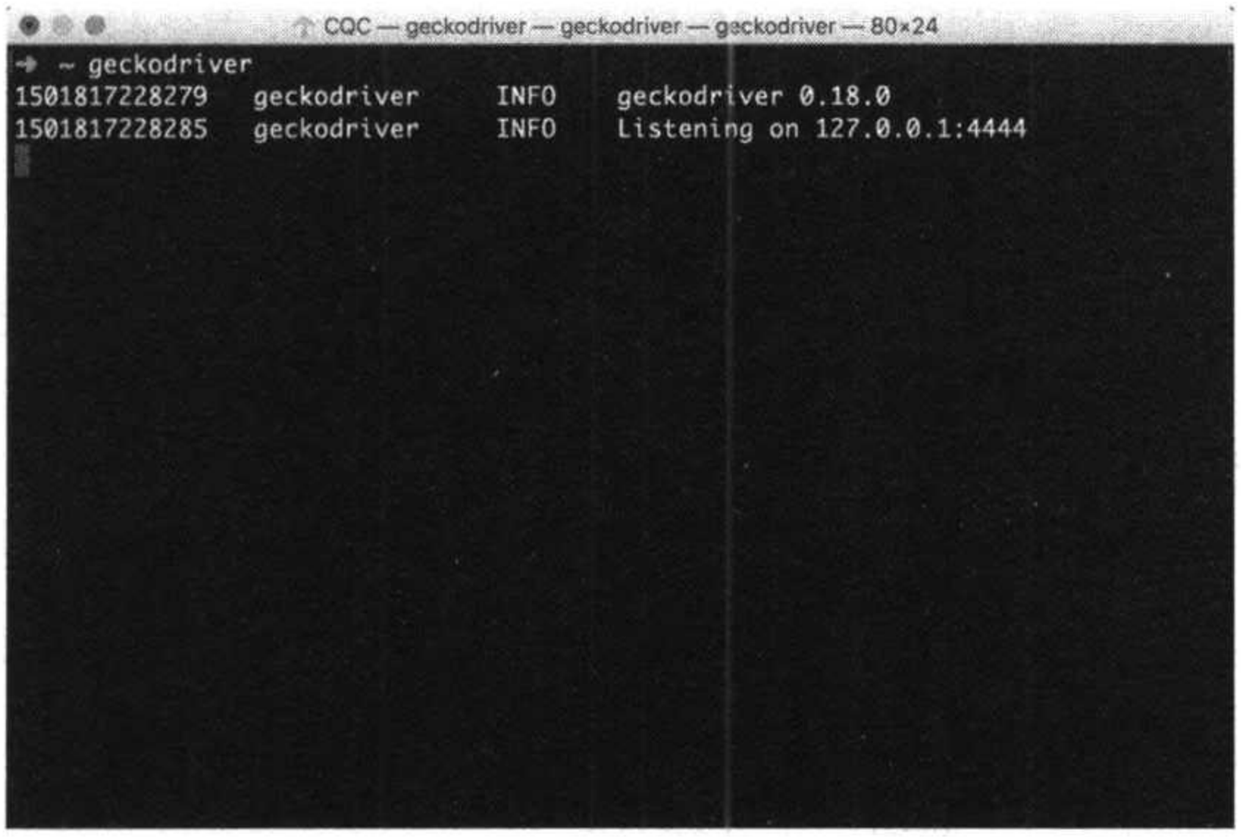


图 1-19 控制台输出

随后执行如下 Python 代码。在程序中测试一下：

```
from selenium import webdriver
browser = webdriver.Firefox()
```

运行之后，若弹出一个空白的 Firefox 浏览器，则证明所有的配置都没有问题；如果没有弹出，请检查之前的每一步配置。

如果没有问题，接下来就可以利用 Firefox 配合 Selenium 来做网页抓取了。

6. 结语

现在我们就可以使用 Chrome 或 Firefox 进行网页抓取了，但是这样可能有个不方便之处：因为程序运行过程中需要一直开着浏览器，在爬取网页的过程中浏览器可能一直动来动去。目前最新的 Chrome 浏览器版本已经支持无界面模式了，但如果版本较旧的话，就不支持。所以这里还有另一种选择，那就是安装一个无界面浏览器 PhantomJS，此时抓取过程会在后台运行，不会再有窗口出现。在下一节中，我们就来了解一下 PhantomJS 的相关安装方法。

1.2.5 PhantomJS 的安装

PhantomJS 是一个无界面的、可脚本编程的 WebKit 浏览器引擎，它原生支持多种 Web 标准：DOM 操作、CSS 选择器、JSON、Canvas 以及 SVG。

Selenium 支持 PhantomJS，这样在运行的时候就不会再弹出一个浏览器了。而且 PhantomJS 的运行效率也很高，还支持各种参数配置，使用非常方便。下面我们就来了解一下 PhantomJS 的安装过程。

1. 相关链接

- 官方网站：http://phantomjs.org
- 官方文档：http://phantomjs.org/quick-start.html
- 下载地址：http://phantomjs.org/download.html
- API 接口说明：http://phantomjs.org/api/command-line.html

2. 下载 PhantomJS

我们需要在官方网站下载对应的安装包，PhantomJS 支持多种操作系统，比如 Windows、Linux、Mac、FreeBSD 等，我们可以选择对应的平台并将安装包下载下来。

下载完成后，将 PhantomJS 可执行文件所在的路径配置到环境变量里。比如在 Windows 下，将下载的文件解压之后并打开，会看到一个 bin 文件夹，里面包括一个可执行文件 phantomjs.exe，我们需要将它直接放在配置好环境变量的路径下或者将它所在的路径配置到环境变量里。比如，我们既可以将它直接复制到 Python 的 Scripts 文件夹，也可以将它所在的 bin 目录加入到环境变量。

Windows 下环境变量的配置可以参见 1.1 节，Linux 及 Mac 环境变量的配置可以参见 1.2.3 节，在此不再赘述，关键在于将 PhantomJS 的可执行文件所在路径配置到环境变量里。

配置成功后，可以在命令行下测试一下，输入：

```
phantomjs
```

如果可以进入到 PhantomJS 的命令行，那就证明配置完成了，如图 1-20 所示。

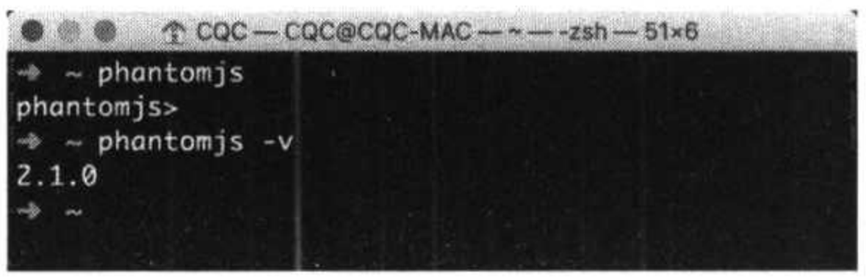


图 1-20　控制台

3. 验证安装

在 Selenium 中使用的话，我们只需要将 Chrome 切换为 PhantomJS 即可：

```
from selenium import webdriver
browser = webdriver.PhantomJS()
browser.get('https://www.baidu.com')
print(browser.current_url)
```

运行之后，我们就不会发现有浏览器弹出了，但实际上 PhantomJS 已经运行起来了。这里我们访问了百度，然后将当前的 URL 打印出来。

控制台的输出如下：

```
https://www.baidu.com/
```

如此一来，我们便完成了 PhantomJS 的配置，后面可以利用它来完成一些页面的抓取。

这里我们介绍了 Selenium 对应的三大主流浏览器的对接方式，后面我们会对 Selenium 及各个浏览器的对接方法进行更加深入的探究。

1.2.6　aiohttp 的安装

之前介绍的 requests 库是一个阻塞式 HTTP 请求库，当我们发出一个请求后，程序会一直等待服务器响应，直到得到响应后，程序才会进行下一步处理。其实，这个过程比较耗费时间。如果程序可以在这个等待过程中做一些其他的事情，如进行请求的调度、响应的处理等，那么爬取效率一定会大大提高。

aiohttp就是这样一个提供异步Web服务的库，从Python 3.5版本开始，Python中加入了async/await关键字，使得回调的写法更加直观和人性化。aiohttp的异步操作借助于async/await关键字的写法变得更加简洁，架构更加清晰。使用异步请求库进行数据抓取时，会大大提高效率，下面我们来看一下这个库的安装方法。

1. 相关链接

- 官方文档：http://aiohttp.readthedocs.io/en/stable
- GitHub：https://github.com/aio-libs/aiohttp
- PyPI：https://pypi.python.org/pypi/aiohttp

2. pip安装

这里推荐使用pip安装，命令如下：

```
pip3 install aiohttp
```

另外，官方还推荐安装如下两个库：一个是字符编码检测库cchardet，另一个是加速DNS的解析库aiodns。安装命令如下：

```
pip3 install cchardet aiodns
```

3. 测试安装

安装完成之后，可以在Python命令行下测试：

```
$ python3
>>> import aiohttp
```

如果没有错误报出，则证明库已经安装好了。

4. 结语

我们会在后面的实例中用到这个库，比如维护一个代理池时，利用异步方式检测大量代理的运行状况，会极大地提升效率。

1.3 解析库的安装

抓取网页代码之后，下一步就是从网页中提取信息。提取信息的方式有多种多样，可以使用正则来提取，但是写起来相对比较烦琐。这里还有许多强大的解析库，如lxml、Beautiful Soup、pyquery等。此外，还提供了非常强大的解析方法，如XPath解析和CSS选择器解析等，利用它们，我们可以高效便捷地从网页中提取有效信息。

本节中，我们就来介绍一下这些库的安装过程。

1.3.1 lxml的安装

lxml是Python的一个解析库，支持HTML和XML的解析，支持XPath解析方式，而且解析效率非常高。本节中，我们了解一下lxml的安装方式，这主要从Windows、Linux和Mac三大平台来介绍。

1. 相关链接

- 官方网站：http://lxml.de
- GitHub：https://github.com/lxml/lxml
- PyPI：https://pypi.python.org/pypi/lxml

2. Windows 下的安装

在 Windows 下，可以先尝试利用 pip 安装，此时直接执行如下命令即可：

```
pip3 install lxml
```

如果没有任何报错，则证明安装成功。

如果出现报错，比如提示缺少 libxml2 库等信息，可以采用 wheel 方式安装。

推荐直接到这里（链接为：http://www.lfd.uci.edu/~gohlke/pythonlibs/#lxml）下载对应的 wheel 文件，找到本地安装 Python 版本和系统对应的 lxml 版本，例如 Windows 64 位、Python 3.6，就选择 lxml-3.8.0-cp36-cp36m-win_amd64.whl，将其下载到本地。

然后利用 pip 安装即可，命令如下：

```
pip3 install lxml-3.8.0-cp36-cp36m-win_amd64.whl
```

这样我们就可以成功安装 lxml 了。

3. Linux 下的安装

在 Linux 平台下安装问题不大，同样可以先尝试 pip 安装，命令如下：

```
pip3 install lxml
```

如果报错，可以尝试下面的解决方案。

- **CentOS、Red Hat**

对于此类系统，报错主要是因为缺少必要的库。

执行如下命令安装所需的库即可：

```
sudo yum groupinstall -y development tools
sudo yum install -y epel-release libxslt-devel libxml2-devel openssl-devel
```

主要是 libxslt-devel 和 libxml2-devel 这两个库，lxml 依赖它们。安装好之后，重新尝试 pip 安装即可。

- **Ubuntu、Debian 和 Deepin**

在这些系统下，报错的原因同样可能是缺少了必要的类库，执行如下命令安装：

```
sudo apt-get install -y python3-dev build-essential libssl-dev libffi-dev libxml2 libxml2-dev libxslt1-dev
    zlib1g-dev
```

安装好之后，重新尝试 pip 安装即可。

4. Mac 下的安装

在 Mac 平台下，仍然可以首先尝试 pip 安装，命令如下：

```
pip3 install lxml
```

如果产生错误，可以执行如下命令将必要的类库安装：

```
xcode-select --install
```

之后再重新尝试 pip 安装，就没有问题了。

lxml 是一个非常重要的库，后面的 Beautiful Soup、Scrapy 框架都需要用到此库，所以请一定安装成功。

5. 验证安装

安装完成之后，可以在 Python 命令行下测试：

```
$ python3
>>> import lxml
```

如果没有错误报出，则证明库已经安装好了。

1.3.2 Beautiful Soup 的安装

Beautiful Soup 是 Python 的一个 HTML 或 XML 的解析库，我们可以用它来方便地从网页中提取数据。它拥有强大的 API 和多样的解析方式，本节就来了解下它的安装方式。

1. 相关链接

- 官方文档：https://www.crummy.com/software/BeautifulSoup/bs4/doc
- 中文文档：https://www.crummy.com/software/BeautifulSoup/bs4/doc.zh
- PyPI：https://pypi.python.org/pypi/beautifulsoup4

2. 准备工作

Beautiful Soup 的 HTML 和 XML 解析器是依赖于 lxml 库的，所以在此之前请确保已经成功安装好了 lxml 库，具体的安装方式参见上节。

3. pip 安装

目前，Beautiful Soup 的最新版本是 4.*x* 版本，之前的版本已经停止开发了。这里推荐使用 pip 来安装，安装命令如下：

```
pip3 install beautifulsoup4
```

命令执行完毕之后即可完成安装。

4. wheel 安装

当然，我们也可以从 PyPI 下载 wheel 文件安装，链接如下：https://pypi.python.org/pypi/beautifulsoup4

然后使用 pip 安装 wheel 文件即可。

5. 验证安装

安装完成之后，可以运行下面的代码验证一下：

```
from bs4 import BeautifulSoup
soup = BeautifulSoup('<p>Hello</p>', 'lxml')
print(soup.p.string)
```

运行结果如下：

```
Hello
```

如果运行结果一致，则证明安装成功。

注意，这里我们虽然安装的是 beautifulsoup4 这个包，但是在引入的时候却是 bs4。这是因为这个包源代码本身的库文件夹名称就是 bs4，所以安装完成之后，这个库文件夹就被移入到本机 Python3 的 lib 库里，所以识别到的库文件名就叫作 bs4。

因此，包本身的名称和我们使用时导入的包的名称并不一定是一致的。

1.3.3　pyquery 的安装

pyquery 同样是一个强大的网页解析工具，它提供了和 jQuery 类似的语法来解析 HTML 文档，支持 CSS 选择器，使用非常方便。本节中，我们就来了解一下它的安装方式。

1. 相关链接

- GitHub：https://github.com/gawel/pyquery
- PyPI：https://pypi.python.org/pypi/pyquery
- 官方文档：http://pyquery.readthedocs.io

2. pip 安装

这里推荐使用 pip 安装，命令如下：

```
pip3 install pyquery
```

命令执行完毕之后即可完成安装。

3. wheel 安装

当然，我们也可以到 PyPI（https://pypi.python.org/pypi/pyquery/#downloads）下载对应的 wheel 文件安装。比如如果当前版本为 1.2.17，则下载的文件名称为 pyquery-1.2.17-py2.py3-none-any.whl，此时下载到本地再进行 pip 安装即可，命令如下：

```
pip3 install pyquery-1.2.17-py2.py3-none-any.whl
```

4. 验证安装

安装完成之后，可以在 Python 命令行下测试：

```
$ python3
>>> import pyquery
```

如果没有错误报出，则证明库已经安装好了。

1.3.4　tesserocr 的安装

在爬虫过程中，难免会遇到各种各样的验证码，而大多数验证码还是图形验证码，这时候我们可以直接用 OCR 来识别。

1. OCR

OCR，即 Optical Character Recognition，光学字符识别，是指通过扫描字符，然后通过其形状将其翻译成电子文本的过程。对于图形验证码来说，它们都是一些不规则的字符，这些字符确实是由字符稍加扭曲变换得到的内容。

例如，对于如图 1-21 和图 1-22 所示的验证码，我们可以使用 OCR 技术来将其转化为电子文本，然后爬虫将识别结果提交给服务器，便可以达到自动识别验证码的过程。



图 1-21 验证码



图 1-22 验证码

tesserocr 是 Python 的一个 OCR 识别库，但其实是对 tesseract 做的一层 Python API 封装，所以它的核心是 tesseract。因此，在安装 tesserocr 之前，我们需要先安装 tesseract。

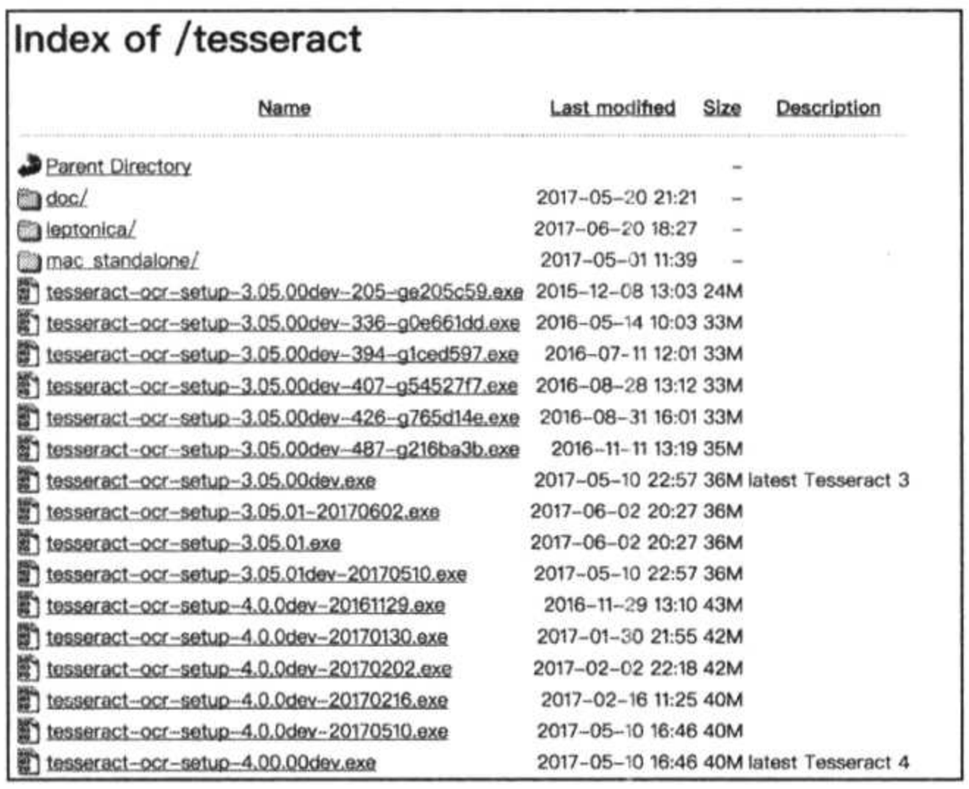
2. 相关链接

- tesserocr GitHub：https://github.com/sirfz/tesserocr
- tesserocr PyPI：https://pypi.python.org/pypi/tesserocr
- tesseract 下载地址：http://digi.bib.uni-mannheim.de/tesseract
- tesseract GitHub：https://github.com/tesseract-ocr/tesseract
- tesseract 语言包：https://github.com/tesseract-ocr/tessdata
- tesseract 文档：https://github.com/tesseract-ocr/tesseract/wiki/Documentation

3. Windows 下的安装

在 Windows 下，首先需要下载 tesseract，它为 tesserocr 提供了支持。

进入下载页面，可以看到有各种.exe 文件的下载列表，这里可以选择下载 3.0 版本。图 1-23 所示为 3.05 版本。



Index of /tesseract

Name	Last modified	Size	Description
Parent Directory		-	
doc/	2017-05-20 21:21	-	
leptonica/	2017-06-20 18:27	-	
mac_standalone/	2017-05-01 11:39	-	
tesseract-ocr-setup-3.05.00dev-205-ge205c59.exe	2015-12-08 13:03	24M	
tesseract-ocr-setup-3.05.00dev-336-g0e661dd.exe	2016-05-14 10:03	33M	
tesseract-ocr-setup-3.05.00dev-394-g1ced597.exe	2016-07-11 12:01	33M	
tesseract-ocr-setup-3.05.00dev-407-g54527f7.exe	2016-08-28 13:12	33M	
tesseract-ocr-setup-3.05.00dev-426-g765d14e.exe	2016-08-31 16:01	33M	
tesseract-ocr-setup-3.05.00dev-487-g216ba3b.exe	2016-11-11 13:19	35M	
tesseract-ocr-setup-3.05.00dev.exe	2017-05-10 22:57	36M	latest Tesseract 3
tesseract-ocr-setup-3.05.01-20170602.exe	2017-06-02 20:27	36M	
tesseract-ocr-setup-3.05.01.exe	2017-06-02 20:27	36M	
tesseract-ocr-setup-3.05.01dev-20170510.exe	2017-05-10 22:57	36M	
tesseract-ocr-setup-4.0.0dev-20161129.exe	2016-11-29 13:10	43M	
tesseract-ocr-setup-4.0.0dev-20170130.exe	2017-01-30 21:55	42M	
tesseract-ocr-setup-4.0.0dev-20170202.exe	2017-02-02 22:18	42M	
tesseract-ocr-setup-4.0.0dev-20170216.exe	2017-02-16 11:25	40M	
tesseract-ocr-setup-4.0.0dev-20170510.exe	2017-05-10 16:46	40M	
tesseract-ocr-setup-4.00.00dev.exe	2017-05-10 16:46	40M	latest Tesseract 4

图 1-23 下载页面

其中文件名中带有 dev 的为开发版本，不带 dev 的为稳定版本，可以选择下载不带 dev 的版本，例如可以选择下载 tesseract-ocr-setup-3.05.01.exe。

下载完成后双击，此时会出现如图 1-24 所示的页面。

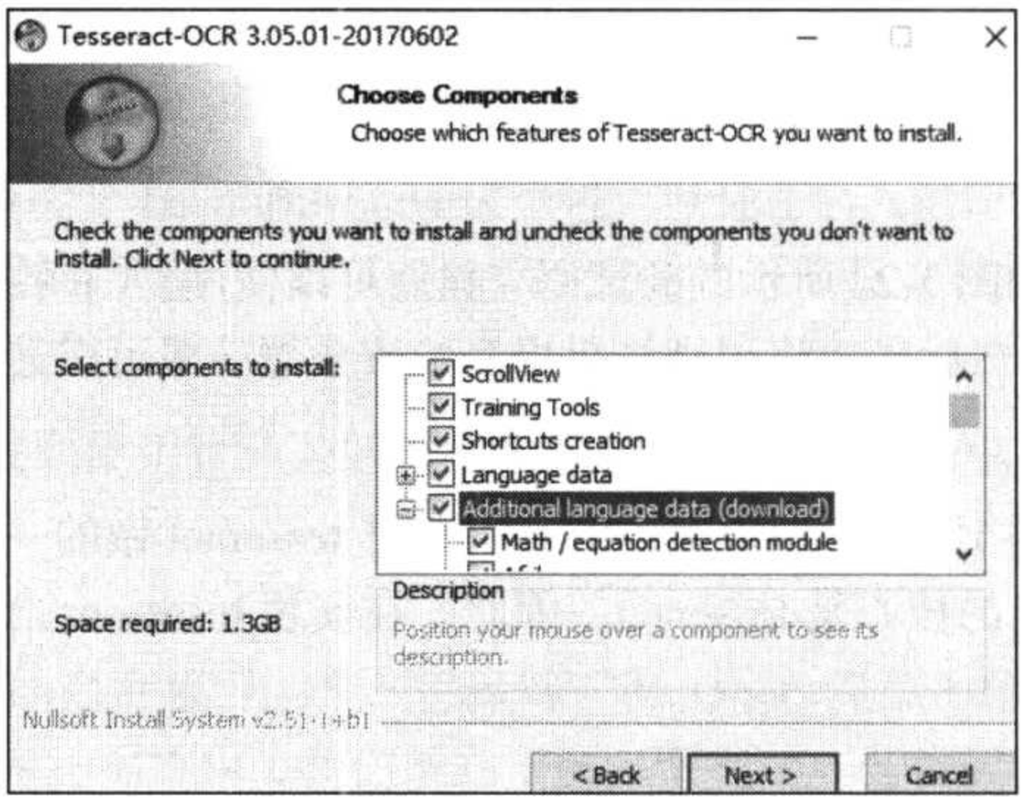


图 1-24　安装页面

此时可以勾选 Additional language data(download)选项来安装 OCR 识别支持的语言包，这样 OCR 便可以识别多国语言。然后一路点击 Next 按钮即可。

接下来，再安装 tesserocr 即可，此时直接使用 pip 安装：

```
pip3 install tesserocr pillow
```

4. Linux 下的安装

对于 Linux 来说，不同系统已经有了不同的发行包了，它可能叫作 tesseract-ocr 或者 tesseract，直接用对应的命令安装即可。

- **Ubuntu、Debian 和 Deepin**

在 Ubuntu、Debian 和 Deepin 系统下，安装命令如下：

```
sudo apt-get install -y tesseract-ocr libtesseract-dev libleptonica-dev
```

- **CentOS、Red Hat**

在 CentOS 和 Red Hat 系统下，安装命令如下：

```
yum install -y tesseract
```

在不同发行版本运行如上命令，即可完成 tesseract 的安装。

安装完成后，便可以调用 tesseract 命令了。

接着，我们查看一下其支持的语言：

```
tesseract --list-langs
```

运行结果示例：

```
List of available languages (3):
eng
osd
equ
```

结果显示它只支持几种语言，如果想要安装多国语言，还需要安装语言包，官方叫作 tessdata（其下载链接为：https://github.com/tesseract-ocr/tessdata）。

利用 Git 命令将其下载下来并迁移到相关目录即可，不同版本的迁移命令如下所示。

在 Ubuntu、Debian 和 Deepin 系统下的迁移命令如下：

```
git clone https://github.com/tesseract-ocr/tessdata.git
sudo mv tessdata/* /usr/share/tesseract-ocr/tessdata
```

在 CentOS 和 Red Hat 系统下的迁移命令如下：

```
git clone https://github.com/tesseract-ocr/tessdata.git
sudo mv tessdata/* /usr/share/tesseract/tessdata
```

这样就可以将下载下来的语言包全部安装了。

这时我们重新运行列出所有语言的命令：

```
tesseract --list-langs
```

结果如下：

```
List of available languages (107):
afr
amh
ara
asm
aze
aze_cyrl
bel
ben
bod
bos
bul
cat
ceb
ces
chi_sim
chi_tra
...
```

可以发现，这里列出的语言就多了很多，比如 chi_sim 就代表简体中文，这就证明语言包安装成功了。

接下来再安装 tesserocr 即可，这里直接使用 pip 安装：

```
pip3 install tesserocr pillow
```

5. Mac 下的安装

在 Mac 下，我们首先使用 Homebrew 安装 ImageMagick 和 tesseract 库：

```
brew install imagemagick
brew install tesseract --all-languages
```

接下来再安装 tesserocr 即可：

```
pip3 install tesserocr pillow
```

这样我们便完成了 tesserocr 的安装。

6. 验证安装

接下来，我们可以使用 tesseract 和 tesserocr 来分别进行测试。

下面我们以如图 1-25 所示的图片为样例进行测试。



图 1-25　测试样例

该图片的链接为 https://raw.githubusercontent.com/Python3WebSpider/TestTess/master/image.png，可以直接保存或下载。

首先用命令行进行测试，将图片下载下来并保存为 image.png，然后用 tesseract 命令测试：

```
tesseract image.png result -l eng && cat result.txt
```

运行结果如下：

```
Tesseract Open Source OCR Engine v3.05.01 with Leptonica
Python3WebSpider
```

这里我们调用了 tesseract 命令，其中第一个参数为图片名称，第二个参数 result 为结果保存的目标文件名称，-l 指定使用的语言包，在此使用英文（eng）。然后，再用 cat 命令将结果输出。

运行结果便是图片的识别结果：Python3WebSpider。可以看到，这时已经成功将图片文字转为电子文本了。

然后还可以利用 Python 代码来测试，这里就需要借助于 tesserocr 库了，测试代码如下：

```
import tesserocr
from PIL import Image
image = Image.open('image.png')
print(tesserocr.image_to_text(image))
```

我们首先利用 Image 读取了图片文件，然后调用了 tesserocr 的 image_to_text()方法，再将其识别结果输出。

运行结果如下：

```
Python3WebSpider
```

另外，我们还可以直接调用 file_to_text()方法，这可以达到同样的效果：

```
import tesserocr
print(tesserocr.file_to_text('image.png'))
```

运行结果：

```
Python3WebSpider
```

如果成功输出结果，则证明 tesseract 和 tesserocr 都已经安装成功。

1.4　数据库的安装

作为数据存储的重要部分，数据库同样是必不可少的，数据库可以分为关系型数据库和非关系型

数据库。

关系型数据库如 SQLite、MySQL、Oracle、SQL Server、DB2 等，其数据库是以表的形式存储；非关系型数据库如 MongoDB、Redis，它们的存储形式是键值对，存储形式更加灵活。

本书用到的数据库主要有关系型数据库 MySQL 及非关系型数据库 MongoDB、Redis。

本节中，我们来了解一下它们的安装方式。

1.4.1 MySQL 的安装

MySQL 是一个轻量级的关系型数据库，本节中我们来了解下它的安装方式。

1. 相关链接

- 官方网站：https://www.mysql.com/cn
- 下载地址：https://www.mysql.com/cn/downloads
- 中文教程：http://www.runoob.com/mysql/mysql-tutorial.html

2. Windows 下的安装

对于 Windows 来说，可以直接在百度软件中心搜索 MySQL，下载其提供的 MySQL 安装包，速度还是比较快的。

当然，最安全稳妥的方式是直接到官网下载安装包进行安装，但是这样做有个缺点，那就是需要登录才可以下载，而且速度不快。

下载完成后，双击安装包即可安装，这里直接选择默认选项，点击 Next 按钮安装即可。这里需要记住图 1-26 所设置的密码。

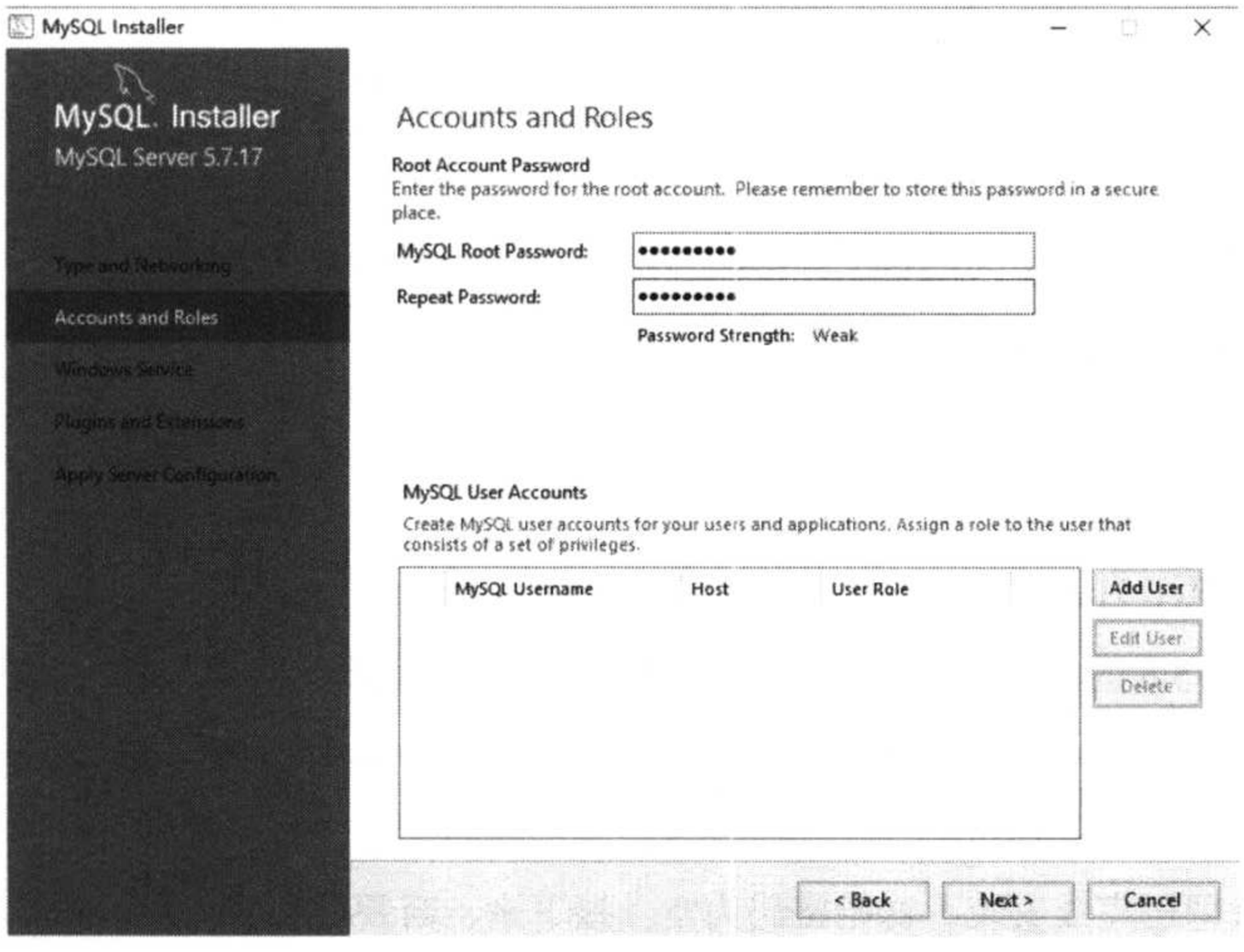


图 1-26 设置密码页面

安装完成后，我们可以在“计算机”→“管理”→“服务”页面开启和关闭 MySQL 服务，如图 1-27 所示。

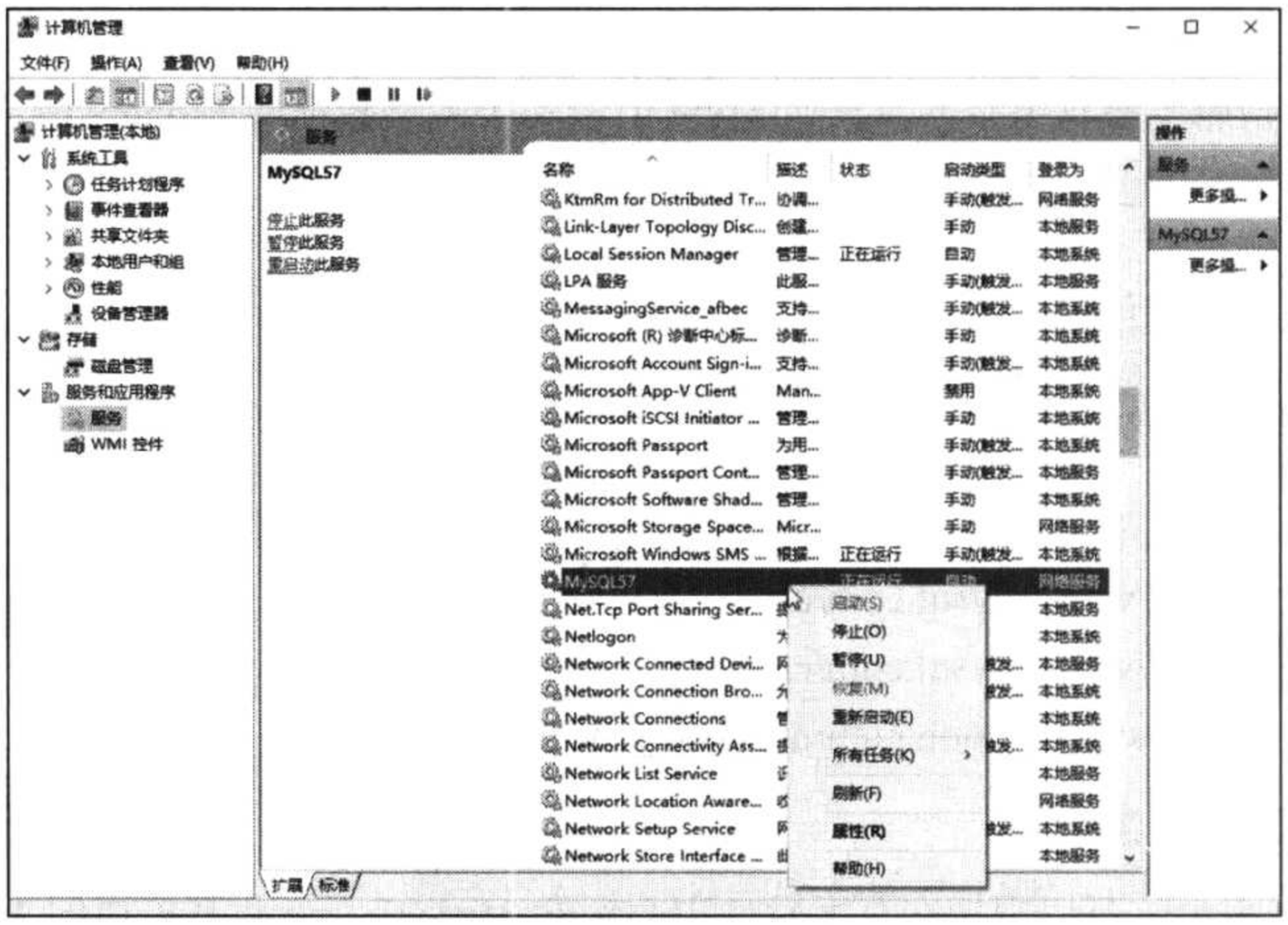


图 1-27　系统服务页面

如果启动了 MySQL 服务，就可以使用它来存储数据了。

3. Linux 下的安装

下面我们仍然分平台来介绍。

- **Ubuntu、Debian 和 Deepin**

在 Ubuntu、Debian 和 Deepin 系统中，我们直接使用 apt-get 命令即可安装 MySQL：

```
sudo apt-get update
sudo apt-get install -y mysql-server mysql-client
```

在安装过程中，会提示输入用户名和密码，输入后等待片刻即可完成安装。

启动、关闭和重启 MySQL 服务的命令如下：

```
sudo service mysql start
sudo service mysql stop
sudo service mysql restart
```

- **CentOS 和 Red Hat**

这里以 MySQL 5.6 的 Yum 源为例来说明（如果需要更高版本，可以另寻），安装命令如下：

```
wget http://repo.mysql.com/mysql-community-release-el7-5.noarch.rpm
sudo rpm -ivh mysql-community-release-el7-5.noarch.rpm
yum install -y mysql mysql-server
```

运行如上命令即可完成安装，初始密码为空。接下来，需要启动 MySQL 服务。

启动 MySQL 服务的命令如下：

```
sudo systemctl start mysqld
```

停止、重启 MySQL 服务的命令如下：

```
sudo systemctl stop mysqld
sudo systemctl restart mysqld
```

上面我们完成了 Linux 下 MySQL 的安装，之后可以修改密码，此时可以执行如下命令：

```
mysql -uroot -p
```

输入密码后，进入 MySQL 命令行模式，接着输入如下命令：

```
use mysql;
UPDATE user SET Password = PASSWORD('newpass') WHERE user = 'root';
FLUSH PRIVILEGES;
```

其中 newpass 为修改的新的 MySQL 密码，请自行替换。

由于 Linux 一般会作为服务器使用，为了使 MySQL 可以被远程访问，我们需要修改 MySQL 的配置文件，配置文件的路径一般为/etc/mysql/my.cnf。

比如，使用 vi 进行修改的命令如下：

```
vi /etc/mysql/my.cnf
```

注释此行：

```
bind-address = 127.0.0.1
```

此行限制了 MySQL 只能本地访问而不能远程访问，注释掉即可解除此限制。

修改完成后重启 MySQL 服务，此时 MySQL 就可以被远程访问了。

到此为止，在 Linux 下安装 MySQL 的过程就结束了。

4. Mac 下的安装

这里推荐使用 Homebrew 安装，直接执行 brew 命令即可：

```
brew install mysql
```

启动、停止和重启 MySQL 服务的命令如下：

```
sudo mysql.server start
sudo mysql.server stop
sudo mysql.server restart
```

Mac 一般不会作为服务器使用，如果想取消本地 host 绑定，那么需要修改 my.cnf 文件，然后重启服务。

1.4.2 MongoDB 的安装

MongoDB 是由 C++语言编写的非关系型数据库，是一个基于分布式文件存储的开源数据库系统，其内容存储形式类似 JSON 对象，它的字段值可以包含其他文档、数组及文档数组，非常灵活。

MongoDB 支持多种平台，包括 Windows、Linux、Mac OS、Solaris 等，在其官方网站（https://www.mongodb.com/download-center）均可找到对应的安装包。

本节中，我们来看下它的安装过程。

1. 相关链接

- 官方网站：https://www.mongodb.com
- 官方文档：https://docs.mongodb.com
- GitHub：https://github.com/mongodb
- 中文教程：http://www.runoob.com/mongodb/mongodb-tutorial.html

2. Windows 下的安装

这里直接在官网（如图 1-28 所示）点击 DOWNLOAD 按钮下载 msi 安装包即可。

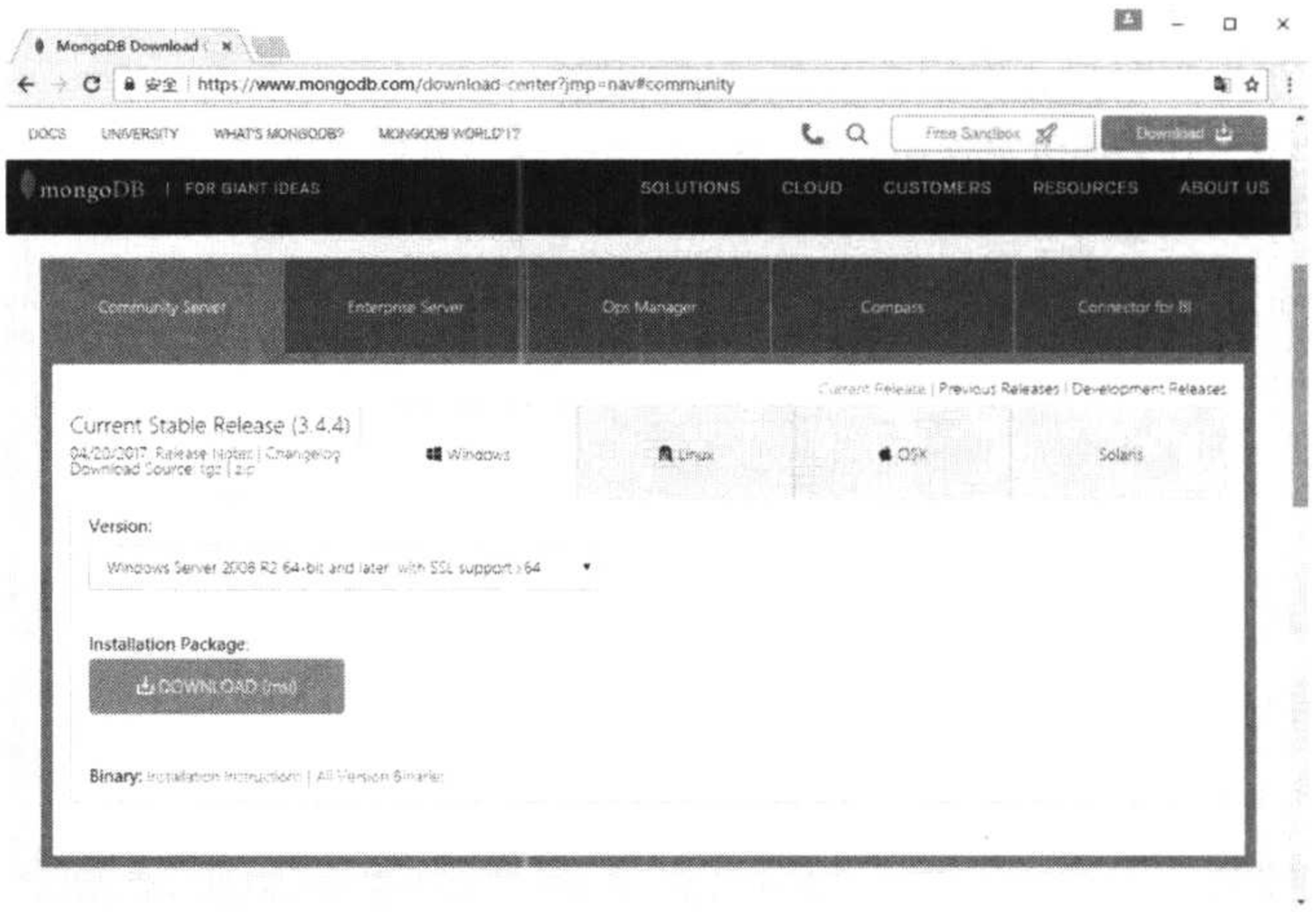


图 1-28　MongoDB 官网

下载完成后，双击它开始安装，指定 MongoDB 的安装路径，例如此处我指定的安装路径为 C:\MongoDB\Server\3.4，如图 1-29 所示。当然，这里也可以自行选择路径。

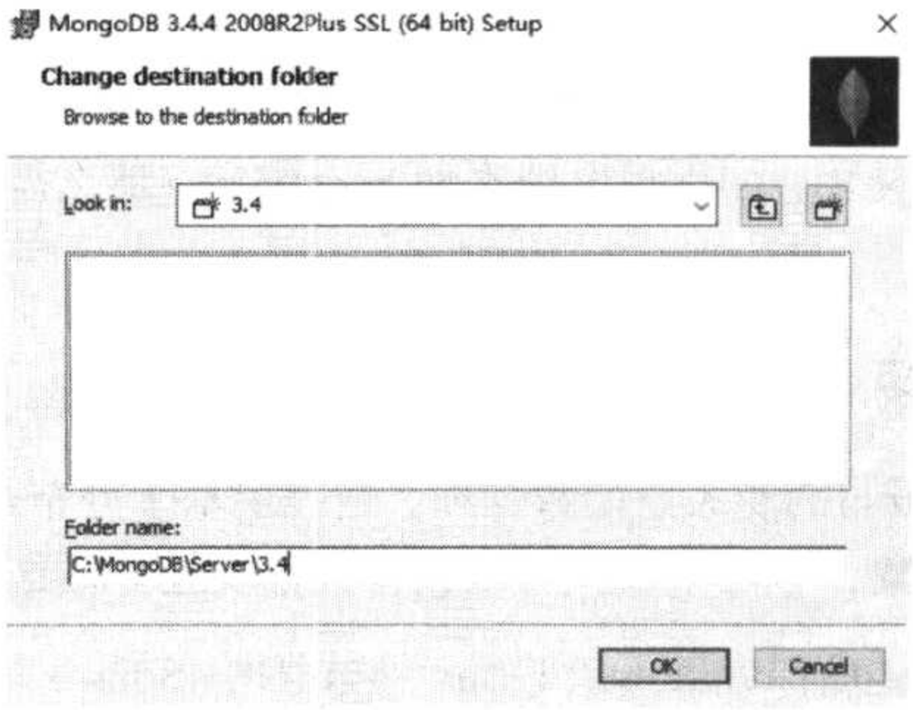


图 1-29　指定安装路径

点击 Next 按钮执行安装即可。

安装成功之后，进入 MongoDB 的安装目录，此处是 C:\MongoDB\Server\3.4，在 bin 目录下新建同级目录 data，如图 1-30 所示。



图 1-30 新建 data 目录

然后进入 data 文件夹，新建子文件夹 db 来存储数据目录，如图 1-31 所示。

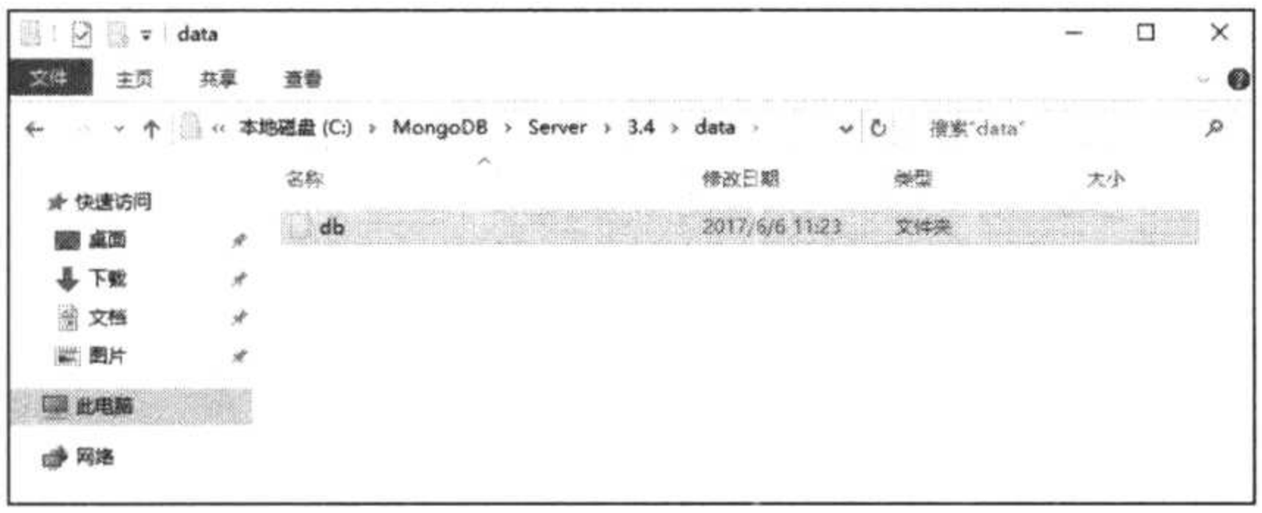


图 1-31 新建 db 目录

之后打开命令行，进入 MongoDB 安装目录的 bin 目录下，运行 MongoDB 服务：

```
mongod --dbpath "C:\MongoDB\Server\3.4\data\db"
```

请记得将此处的路径替换成你的主机 MongoDB 安装路径。

运行之后，会出现一些输出信息，如图 1-32 所示。

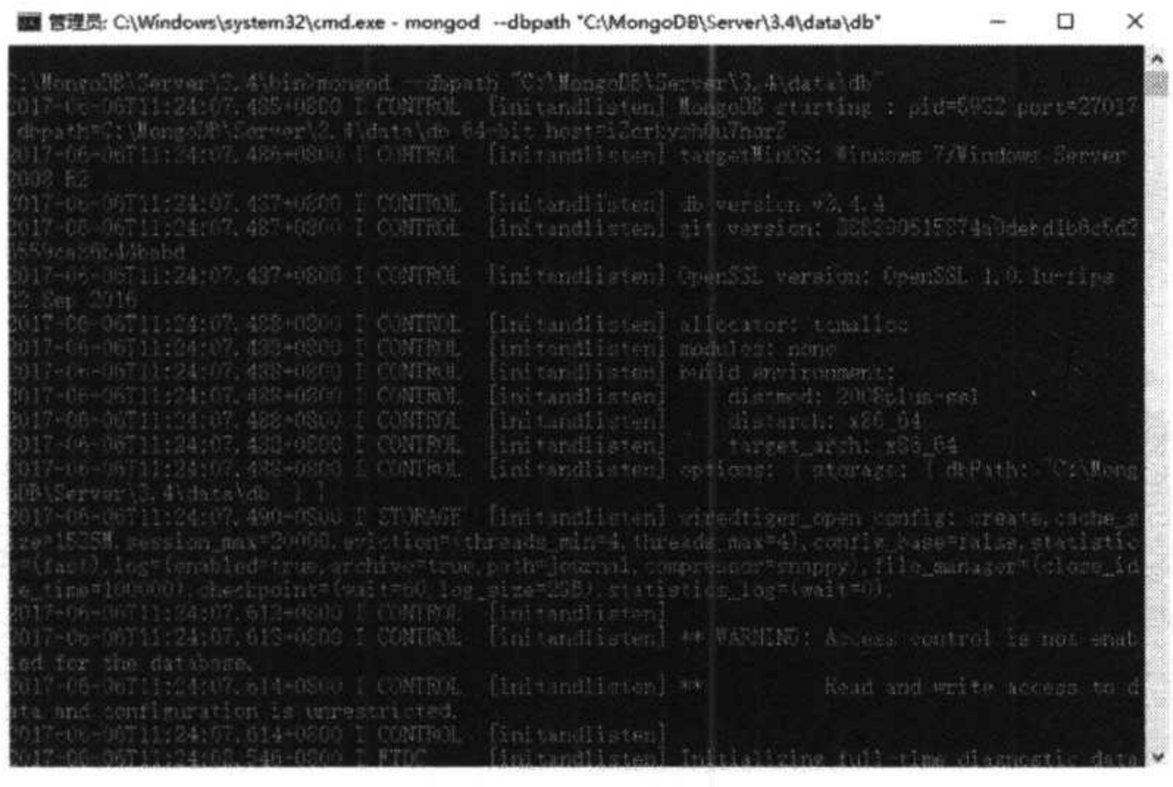


图 1-32 运行结果

这样我们就启动 MongoDB 服务了。

但是如果我们想一直使用 MongoDB，就不能关闭此命令行了。如果意外关闭或重启，MongoDB 服务就不能使用了。这显然不是我们想要的。所以，接下来还需将 MongoDB 配置成系统服务。

首先，以管理员模式运行命令行。注意，此处一定要以管理员身份运行，否则可能配置失败，如图 1-33 所示。

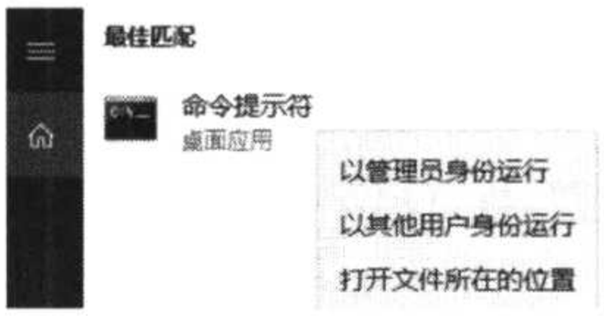


图 1-33　以管理员身份运行

在“开始”菜单中搜索 cmd，找到命令行，然后右击它以管理员身份运行即可。

随后新建一个日志文件，在 bin 目录新建 logs 同级目录，进入之后新建一个 mongodb.log 文件，用于保存 MongoDB 的运行日志，如图 1-34 所示。



图 1-34　新建 mongodb.log 文件

在命令行下输入如下内容：

```
mongod --bind_ip 0.0.0.0 --logpath "C:\MongoDB\Server\3.4\logs\mongodb.log" --logappend --dbpath
    "C:\MongoDB\Server\3.4\data\db" --port 27017 --serviceName "MongoDB" --serviceDisplayName "MongoDB" --install
```

这里的意思是绑定 IP 为 0.0.0.0（即任意 IP 均可访问），指定日志路径、数据库路径和端口，指定服务名称。需要注意的是，这里依然需要把路径替换成你的 MongoDB 安装路径，运行此命令后即可安装服务，运行结果如图 1-35 所示。



图 1-35　运行结果

如果没有出现错误提示，则证明 MongoDB 服务已经安装成功。

可以在服务管理页面查看到系统服务，如图 1-36 所示。

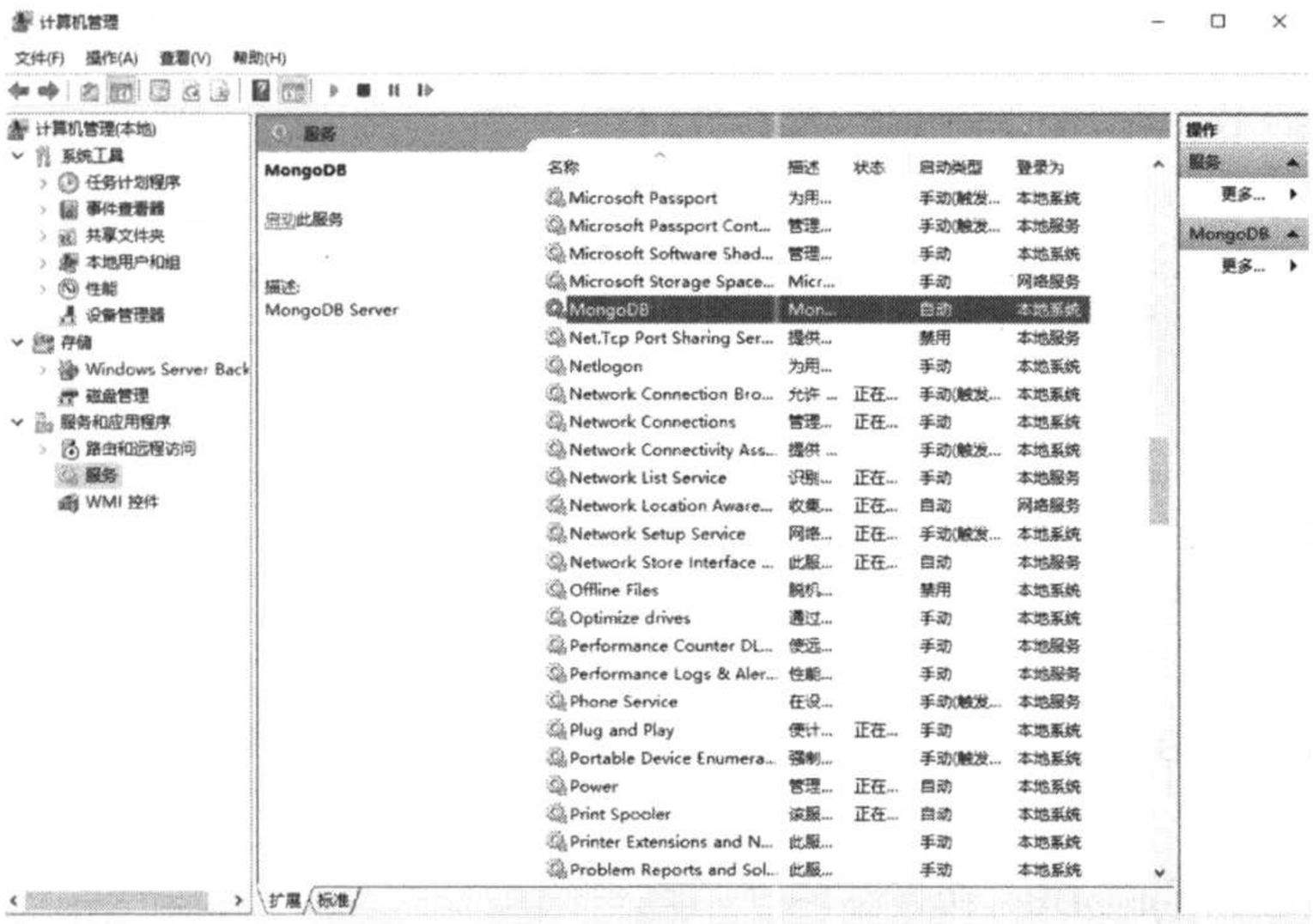


图 1-36 系统服务页面

然后就可以设置它的开机启动方式了，如自动启动或手动启动等，这样我们就可以非常方便地管理 MongoDB 服务了。

启动服务后，在命令行下就可以利用 mongo 命令进入 MongoDB 命令交互环境了，如图 1-37 所示。

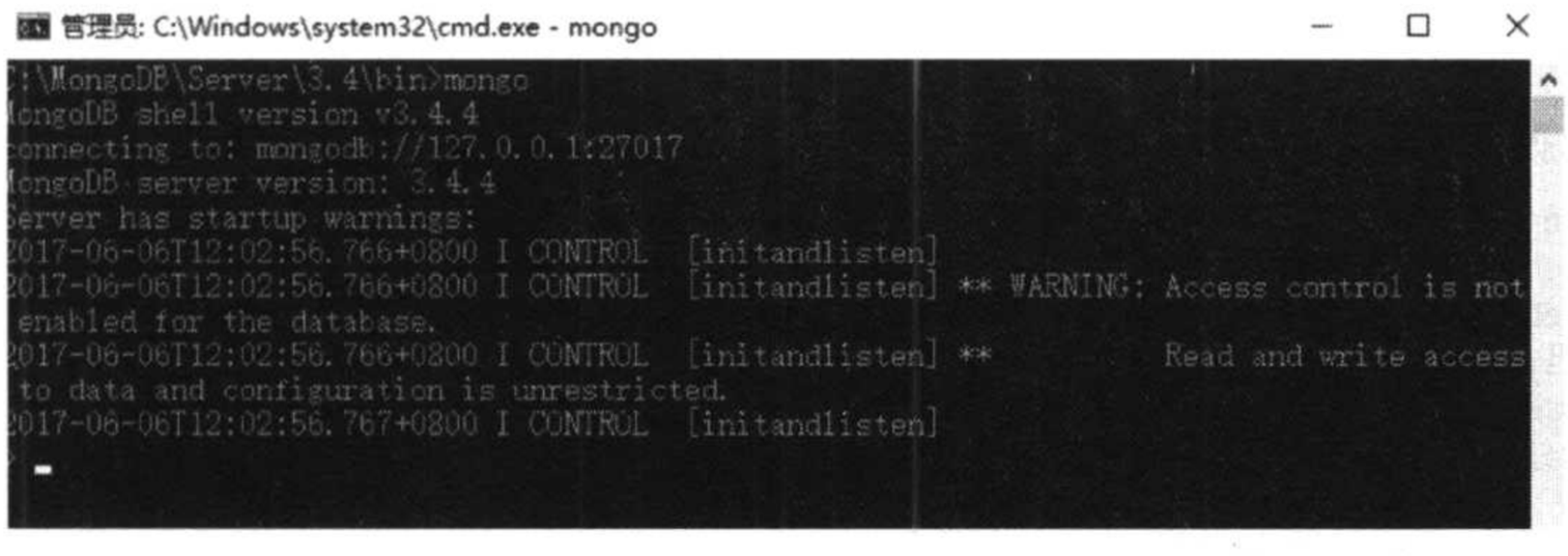


图 1-37 命令行模式

这样，Windows 下的 MongoDB 配置就完成了。

3. Linux 下的安装

这里以 MongoDB 3.4 为例说明 MongoDB 的安装过程。

- **Ubuntu**

首先，导入 MongoDB 的 GPG key：

```
sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 --recv 0C49F3730359A14518585931BC711F9BA15703C6
```

随后创建 apt-get 源列表，各个系统版本对应的命令分别如下。

- Ubuntu 12.04 对应的命令如下：

```
echo "deb [ arch=amd64 ] http://repo.mongodb.org/apt/ubuntu precise/mongodb-org/3.4 multiverse" | sudo tee
    /etc/apt/sources.list.d/mongodb-org-3.4.list
```

- Ubuntu 14.04 对应的命令如下：

```
echo "deb [ arch=amd64 ] http://repo.mongodb.org/apt/ubuntu trusty/mongodb-org/3.4 multiverse" | sudo tee
    /etc/apt/sources.list.d/mongodb-org-3.4.list
```

- Ubuntu 16.04 对应的命令如下：

```
echo "deb [ arch=amd64,arm64 ] http://repo.mongodb.org/apt/ubuntu xenial/mongodb-org/3.4 multiverse" |
    sudo tee /etc/apt/sources.list.d/mongodb-org-3.4.list
```

随后更新 apt-get 源：

```
sudo apt-get update
```

之后安装 MongoDB 即可：

```
sudo apt-get install -y mongodb-org
```

安装完成后运行 MongoDB，命令如下：

```
mongod --port 27017 --dbpath /data/db
```

运行命令之后，MongoDB 就在 27017 端口上运行了，数据文件会保存在/data/db 路径下。

一般情况下，我们在 Linux 上配置 MongoDB 都是为了远程连接使用的，所以这里还需要配置一下 MongoDB 的远程连接以及用户名和密码。

接着，进入 MongoDB 命令行：

```
mongo --port 27017
```

现在我们就已经进入到 MongoDB 的命令行交互模式下了，在此模式下运行如下命令：

```
> use admin
switched to db admin
> db.createUser({user: 'admin', pwd: 'admin123', roles: [{role: 'root', db: 'admin'}]})
Successfully added user: {
        "user" : "admin",
        "roles" : [
            {
                "role" : "root",
                "db" : "admin"
            }
        ]
}
```

这样我们就创建了一个用户名为 admin，密码为 admin123 的用户，赋予最高权限。

随后需要修改 MongoDB 的配置文件，此时执行如下命令：

```
sudo vi /etc/mongod.conf
```

然后修改 net 部分为：

```
net:
  port: 27017
  bindIp: 0.0.0.0
```

这样配置后，MongoDB 可被远程访问。

另外，还需要添加如下的权限认证配置。此时直接添加如下内容到配置文件即可：

```
security:
  authorization: enabled
```

配置完成之后，我们需要重新启动 MongoDB 服务，命令如下：

```
sudo service mongod restart
```

这样远程连接和权限认证就配置完成了。

- **CentOS 和 Red Hat**

首先，添加 MongoDB 源：

```
sudo vi /etc/yum.repos.d/mongodb-org.repo
```

接着修改如下内容并保存：

```
[mongodb-org-3.4]
name=MongoDB Repository
baseurl=https://repo.mongodb.org/yum/redhat/$releasever/mongodb-org/3.4/x86_64/
gpgcheck=1
enabled=1
gpgkey=https://www.mongodb.org/static/pgp/server-3.4.asc
```

然后执行 yum 命令安装：

```
sudo yum install mongodb-org
```

这里启动 MongoDB 服务的命令如下：

```
sudo systemctl start mongod
```

停止和重新加载 MongoDB 服务的命令如下：

```
sudo systemctl stop mongod
sudo systemctl reload mongod
```

有关远程连接和认证配置，可以参考前面，方式是相同的。

更多 Linux 发行版的 MongoDB 安装方式可以参考官方文档：https://docs.mongodb.com/manual/administration/install-on-linux/。

4. Mac 下的安装

这里推荐使用 Homebrew 安装，直接执行 brew 命令即可：

```
brew install mongodb
```

然后创建一个新文件夹/data/db，用于存放 MongoDB 数据。

这里启动 MongoDB 服务的命令如下：

```
brew services start mongodb
sudo mongod
```

停止和重启 MongoDB 服务的命令分别是：

```
brew services stop mongodb
brew services restart mongodb
```

5. 可视化工具

这里推荐一个可视化工具 RoboMongo/Robo 3T，它使用简单，功能强大，官方网站为 https://robomongo.org/，三大平台都支持，下载链接为 https://robomongo.org/download。

另外，还有一个简单易用的可视化工具——Studio 3T，它同样具有方便的图形化管理界面，官方网站为 https://studio3t.com，同样支持三大平台，下载链接为 https://studio3t.com/download/。

1.4.3　Redis 的安装

Redis 是一个基于内存的高效的非关系型数据库，本节中我们来了解一下它在各个平台的安装过程。

1. 相关链接

- 官方网站：https://redis.io
- 官方文档：https://redis.io/documentation
- 中文官网：http://www.redis.cn
- GitHub：https://github.com/antirez/redis
- 中文教程：http://www.runoob.com/redis/redis-tutorial.html
- Redis Desktop Manager：https://redisdesktop.com
- Redis Desktop Manager GitHub：https://github.com/uglide/RedisDesktopManager

2. Windows 下的安装

在 Windows 下，Redis 可以直接到 GitHub 的发行版本里面下载，具体下载地址是 https://github.com/MSOpenTech/redis/releases。

打开下载页面后，会发现有许多发行版本及其安装包，如图 1-38 所示。



图 1-38　下载页面

可以下载 Redis-x64-3.2.100.msi 安装即可。

安装过程比较简单，直接点击 Next 按钮安装即可。安装完成后，Redis 便会启动。

在系统服务页面里，可以观察到多了一个正在运行到 Redis 服务，如图 1-39 所示。

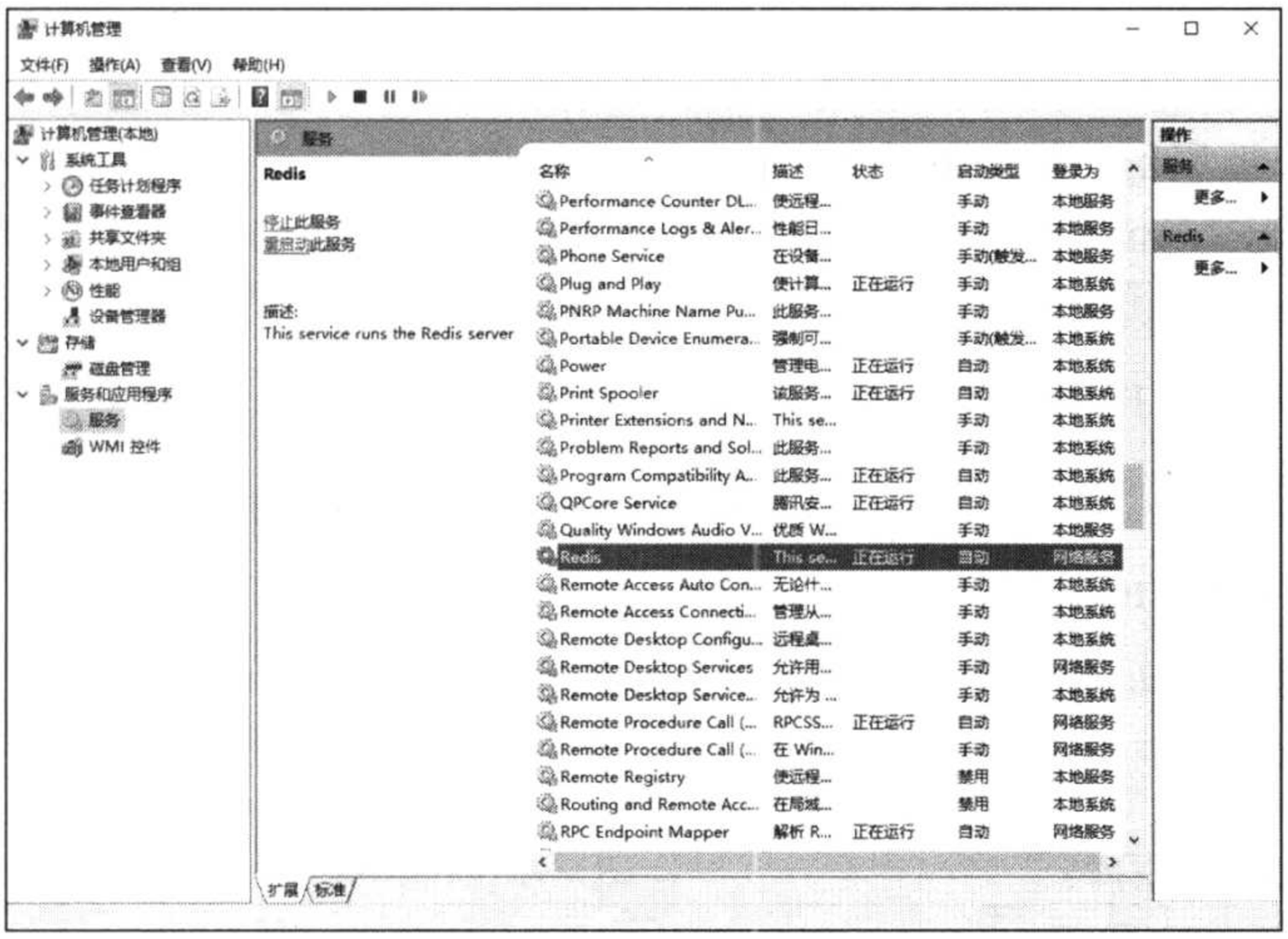


图 1-39 系统服务页面

另外，推荐下载 Redis Desktop Manager 可视化管理工具，来管理 Redis。这既可以到官方网站（链接为：https://redisdesktop.com/download）下载，也可以到 GitHub（链接为：https://github.com/uglide/RedisDesktopManager/releases）下载最新发行版本。

安装后，直接连接本地 Redis 即可。

3. Linux 下的安装

这里依然分为两类平台来介绍。

- **Ubuntu、Debian 和 Deepin**

在 Ubuntu、Debian 和 Deepin 系统下，使用 apt-get 命令安装 Redis：

```
sudo apt-get -y install redis-server
```

然后输入 redis-cli 进入 Redis 命令行模式：

```
$ redis-cli
127.0.0.1:6379> set 'name' 'Germey'
OK
127.0.0.1:6379> get 'name'
"Germey"
```

这样就证明 Redis 成功安装了，但是现在 Redis 还是无法远程连接的，依然需要修改配置文件，配置文件的路径为/etc/redis/redis.conf。

首先，注释这一行：

```
bind 127.0.0.1
```

另外，推荐给 Redis 设置密码，取消注释这一行：

```
requirepass foobared
```

foobared 即当前密码，可以自行修改。

然后重启 Redis 服务，使用的命令如下：

```
sudo /etc/init.d/redis-server restart
```

现在就可以使用密码远程连接 Redis 了。

另外，停止和启动 Redis 服务的命令分别如下：

```
sudo /etc/init.d/redis-server stop
sudo /etc/init.d/redis-server start
```

- **CentOS 和 Red Hat**

在 CentOS 和 Red Hat 系统中，首先添加 EPEL 仓库，然后更新 yum 源：

```
sudo yum install epel-release
sudo yum update
```

然后安装 Redis 数据库：

```
sudo yum -y install redis
```

安装好后启动 Redis 服务即可：

```
sudo systemctl start redis
```

这里同样可以使用 redis-cli 进入 Redis 命令行模式操作。

另外，为了可以使 Redis 能被远程连接，需要修改配置文件，路径为/etc/redis.conf。

参见上文来修改配置文件实现远程连接和密码配置。

修改完成之后保存。

然后重启 Redis 服务即可，命令如下：

```
sudo systemctl restart redis
```

4. Mac 下的安装

这里推荐使用 Homebrew 安装，直接执行如下命令即可：

```
brew install redis
```

启动 Redis 服务的命令如下：

```
brew services start redis
redis-server /usr/local/etc/redis.conf
```

这里同样可以使用 redis-cli 进入 Redis 命令行模式。

在 Mac 下 Redis 的配置文件路径是/usr/local/etc/redis.conf，可以通过修改它来配置访问密码。

修改配置文件后，需要重启 Redis 服务。停止和重启 Redis 服务的命令分别如下：

```
brew services stop redis
brew services restart redis
```

另外，在 Mac 下也可以安装 Redis Desktop Manager 可视化管理工具来管理 Redis。

1.5 存储库的安装

1.4 节中，我们介绍了几个数据库的安装方式，但这仅仅是用来存储数据的数据库，它们提供了存储服务，但如果想要和 Python 交互的话，还需要安装一些 Python 存储库，如 MySQL 需要安装 PyMySQL，MongoDB 需要安装 PyMongo 等。本节中，我们来说明一下这些存储库的安装方式。

1.5.1 PyMySQL 的安装

在 Python 3 中，如果想要将数据存储到 MySQL 中，就需要借助 PyMySQL 来操作，本节中我们介绍一下它的安装方式。

1. 相关链接

- GitHub：https://github.com/PyMySQL/PyMySQL
- 官方文档：http://pymysql.readthedocs.io/
- PyPI：https://pypi.python.org/pypi/PyMySQL

2. pip 安装

这里推荐使用 pip 安装，命令如下：

```
pip3 install pymysql
```

执行完命令后即可完成安装。

3. 验证安装

为了验证库是否已经安装成功，可以在命令行下测试一下。这里首先输入 python3，进入命令行模式，接着输入如下内容：

```
$ python3
>>> import pymysql
>>> pymysql.VERSION
(0, 7, 11, None)
>>>
```

如果成功输出了其版本内容，那么证明 PyMySQL 成功安装。

1.5.2 PyMongo 的安装

在 Python 中，如果想要和 MongoDB 进行交互，就需要借助于 PyMongo 库，这里就来了解一下它的安装方法。

1. 相关链接

- GitHub：https://github.com/mongodb/mongo-python-driver

- 官方文档：https://api.mongodb.com/python/current/
- PyPI：https://pypi.python.org/pypi/pymongo

2. pip 安装

这里推荐使用 pip 安装，命令如下：

```
pip3 install pymongo
```

运行完毕之后，即可完成 PyMongo 的安装。

3. 验证安装

为了验证 PyMongo 库是否已经安装成功，可以在命令行下测试一下：

```
$ python3
>>> import pymongo
>>> pymongo.version
'3.4.0'
>>>
```

如果成功输出了其版本内容，那么证明成功安装。

1.5.3　redis-py 的安装

对于 Redis 来说，我们要使用 redis-py 库来与其交互，这里就来介绍一下它的安装方法。

1. 相关链接

- GitHub：https://github.com/andymccurdy/redis-py
- 官方文档：https://redis-py.readthedocs.io/

2. pip 安装

这里推荐使用 pip 安装，命令如下：

```
pip3 install redis
```

运行完毕之后，即可完成 redis-py 的安装。

3. 验证安装

为了验证 redis-py 库是否已经安装成功，可以在命令行下测试一下：

```
$ python3
>>> import redis
>>> redis.VERSION
(2, 10, 5)
>>>
```

如果成功输出了其版本内容，那么证明成功安装了 redis-py。

1.5.4　RedisDump 的安装

RedisDump 是一个用于 Redis 数据导入/导出的工具，是基于 Ruby 实现的，所以要安装 RedisDump，需要先安装 Ruby。

1. 相关链接

- GitHub：https://github.com/delano/redis-dump
- 官方文档：http://delanotes.com/redis-dump

2. 安装 Ruby

有关 Ruby 的安装方式可以参考 http://www.ruby-lang.org/zh_cn/documentation/installation，这里列出了所有平台的安装方式，可以根据对应的平台选用合适的安装方式。

3. gem 安装

安装完成之后，就可以执行 gem 命令了，它类似于 Python 中的 pip 命令。利用 gem 命令，我们可以安装 RedisDump，具体如下：

```
gem install redis-dump
```

执行完毕之后，即可完成 RedisDump 的安装。

4. 验证安装

安装成功后，就可以执行如下两个命令：

```
redis-dump
redis-load
```

如果可以成功调用，则证明安装成功。

1.6 Web 库的安装

对于 Web，我们应该都不陌生，现在日常访问的网站都是 Web 服务程序搭建而成的。Python 同样不例外，也有一些这样的 Web 服务程序，比如 Flask、Django 等，我们可以拿它来开发网站和接口等。

在本书中，我们主要使用这些 Web 服务程序来搭建一些 API 接口，供我们的爬虫使用。例如，维护一个代理池，代理保存在 Redis 数据库中，我们要将代理池作为一个公共的组件使用，那么如何构建一个方便的平台来供我们获取这些代理呢？最合适不过的就是通过 Web 服务提供一个 API 接口，我们只需要请求接口即可获取新的代理，这样做简单、高效、实用！

书中用到的一些 Web 服务程序主要有 Flask 和 Tornado，这里就分别介绍它们的安装方法。

1.6.1 Flask 的安装

Flask 是一个轻量级的 Web 服务程序，它简单、易用、灵活，这里主要用来做一些 API 服务。

1. 相关链接

- GitHub：https://github.com/pallets/flask
- 官方文档：http://flask.pocoo.org
- 中文文档：http://docs.jinkan.org/docs/flask
- PyPI：https://pypi.python.org/pypi/Flask

2. pip 安装

这里推荐使用 pip 安装，命令如下：

```
pip3 install flask
```

运行完毕后，就完成安装了。

3. 验证安装

安装成功后，可以运行如下实例代码测试一下：

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello World!"

if __name__ == "__main__":
    app.run()
```

可以发现，系统会在 5000 端口开启 Web 服务，控制台输出如下：

```
 * Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

直接访问 http://127.0.0.1:5000/，可以观察到网页中呈现了 Hello World!，如图 1-40 所示，一个最简单的 Flask 程序就运行成功了。

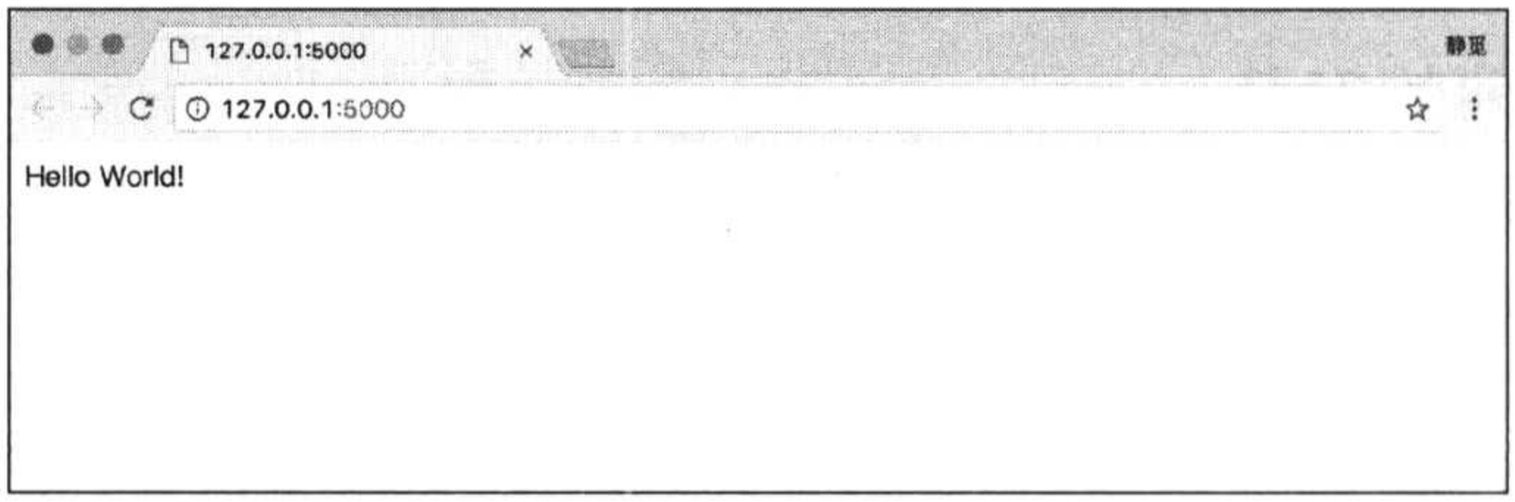


图 1-40　运行结果

4. 结语

后面，我们会利用 Flask+Redis 维护动态代理池和 Cookies 池。

1.6.2　Tornado 的安装

Tornado 是一个支持异步的 Web 框架，通过使用非阻塞 I/O 流，它可以支撑成千上万的开放连接，效率非常高，本节就来介绍一下它的安装方式。

1. 相关链接

- GitHub：https://github.com/tornadoweb/tornado
- PyPI：https://pypi.python.org/pypi/tornado
- 官方文档：http://www.tornadoweb.org

2. pip 安装

这里推荐使用 pip 安装，相关命令如下：

```
pip3 install tornado
```

执行完毕后，即可完成安装。

3. 验证安装

同样，这里也可以用一个 Hello World 程序测试一下，代码如下：

```
import tornado.ioloop
import tornado.web

class MainHandler(tornado.web.RequestHandler):
    def get(self):
        self.write("Hello, world")

def make_app():
    return tornado.web.Application([
        (r"/", MainHandler),
    ])

if __name__ == "__main__":
    app = make_app()
    app.listen(8888)
    tornado.ioloop.IOLoop.current().start()
```

直接运行程序，可以发现系统在 8888 端口运行了 Web 服务，控制台没有输出内容，此时访问 http://127.0.0.1:8888/，可以观察到网页中呈现了 Hello,world，如图 1-41 所示，这就说明 Tornado 成功安装了。

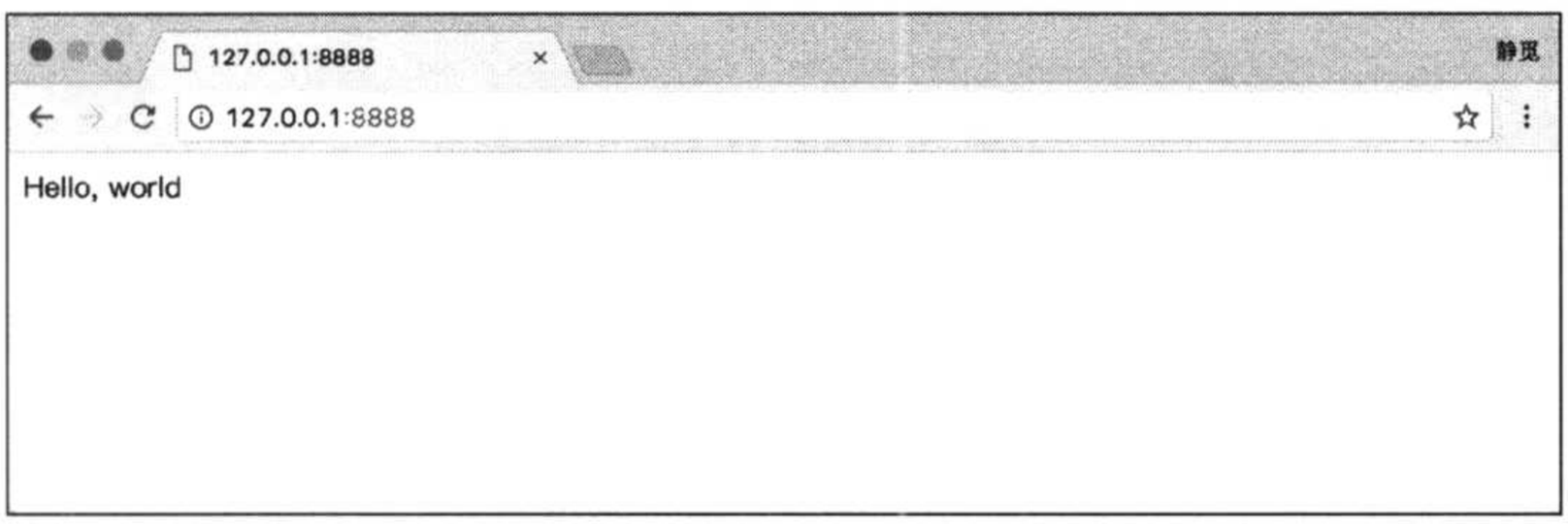


图 1-41　运行结果

4. 结语

后面，我们会利用 Tornado+Redis 来搭建一个 ADSL 拨号代理池。

1.7　App 爬取相关库的安装

除了 Web 网页，爬虫也可以抓取 App 的数据。App 中的页面要加载出来，首先需要获取数据，而这些数据一般是通过请求服务器的接口来获取的。由于 App 没有浏览器这种可以比较直观地看到后台请求的工具，所以主要用一些抓包技术来抓取数据。

有关此电子书试读版的说明

本人可以帮助你找到你要的高清 PDF 电子书，计算机类，文学，艺术，设计，医学，理学，经济，金融等等。

质量都很清晰，为方便读者阅读观看，每本 100°/。都带可跳转的书签索引和目录。

一般情况下，出版半年左右就会有 PDF 极个别的书出 PDF 时间要长一些

如看到试读版信息.说明已经有完整版，需求完整版即可联系我。
请添加 QQ 312082710 和微信312082710

或扫描微信二维码添加



PDF 代找说明：

本人已经帮助了上万人找到了他们需要的 **PDF** ，其实网上有很多 **PDF**,

大家如果在网上不到的话，可以联系我**QQ** 312082710或微信 312082710

大部分我都可以找到，而且每本 100%带书签索引目录。
因 **PDF** 电子书都有版权，请不要随意传播，如果您有经济购买能力，请尽量购买正版。

提供各种书籍的高清 **PDF** 电子版代找服务，如果你找不到自己想要的书的 **PDF** 电子版，我们 可以帮您找到，如有需要，请联系 **QQ** 2028969416 微信 2028969416 备用 **QQ** 202896416
若以上联系方式失效，您可通过以下电子邮件获取有效联系方式。邮箱：312082710@qq.com

若您没有 **QQ** 通讯工具，请点击下面链接与客服取得联系。&

https://item.taobao.com/item.htm?id=565681036651

声明：本人只提供代找服务，每本 100%索弓|书签和目录，因寻找和后期制作 **pdf** 电子书有一定难度
仅收取代找费用。如因 **PDF** 产生的版权纠纷，与本人无关，我们仅仅只是帮助你寻找到你要的 **PDF** 而已。